# LibanPost eShop — administrator guide

# Contents


Contents . . . 4
The dashboard . . . 4
Products . . . 5
Bulk import . . . 8
Categories . . . 9
Inventory . . . 12
Orders . . . 13
Customers . . . 17
Returns . . . 18
Coupons . . . 21
Tax classes . . . 23
Shops . . . 24
Payouts . . . 26
Marketing analytics . . . 29
Reviews moderation . . . 29
Email outbox . . . 31
Exports . . . 33
Audit trail . . . 33
Where the store's own log lives . . . 35
Questions staff usually ask . . . 35
Additional screens . . . 36

**Admin form and field reference . . . 44**

5. Admin area (/Admin) . . . 44
Dashboard — /Admin/Dashboard (Dashboard/Index) . . . 44
Products list — /Admin/Products (Products/Index) . . . 44
Product edit — /Admin/Products/Edit (and Create) (Products/Edit / Save) . . . 44
Bulk import — /Admin/Import (Import/Index) . . . 44
Categories — /Admin/Categories (Categories/Index, Save, Delete) . . . 45
Inventory — /Admin/Inventory (Inventory/Index, Adjust, SetStock) . . . 45
Orders list — /Admin/Orders (Orders/Index) . . . 45
Order detail — /Admin/Orders/Details/{id} (Orders/Details) . . . 45
Returns queue — /Admin/Returns (Returns/Index) . . . 45
Return detail — /Admin/Returns/Details/{id} (Returns/Details) . . . 45
Customers list — /Admin/Customers (Customers/Index) . . . 46
Customer detail — /Admin/Customers/Details/{id} (Customers/Details) . . . 46
Coupons list — /Admin/Coupons (Coupons/Index) . . . 46

Coupon create — /Admin/Coupons/Create (Coupons/Create) . . . . . 46
Coupon edit — /Admin/Coupons/Edit/{id} (Coupons/Edit) . . . . . 46
Tax classes — /Admin/Tax (Tax/Index, Create, Deactivate) . . . . . 47
Shops — /Admin/Shops (Shops/Index) . . . . . 47
Payouts list — /Admin/Payouts (Payouts/Index) . . . . . 47
Payout detail — /Admin/Payouts/Details/{id} (Payouts/Details) . . . . . 47
Marketing — /Admin/Marketing (Marketing/Index) . . . . . 47
Reviews moderation — /Admin/Reviews (Reviews/Index) . . . . . 47
Audit trail — /Admin/Audit (Audit/Index, Export) . . . . . 48
Emails outbox — /Admin/Emails (Emails/Index, Details, Resend, SendTest) . . . . . 48

**Common buttons and what they do (plain language) . . . . . 49**

For store staff. Everything behind `/Admin`: running the catalogue, handling orders and customers, moderating reviews, settling vendors, reading the analytics, and reading the store's own log.

> Sign in at `/Account` with a staff account, then go to `/Admin`. The demonstration store's admin > account is documented in the README.

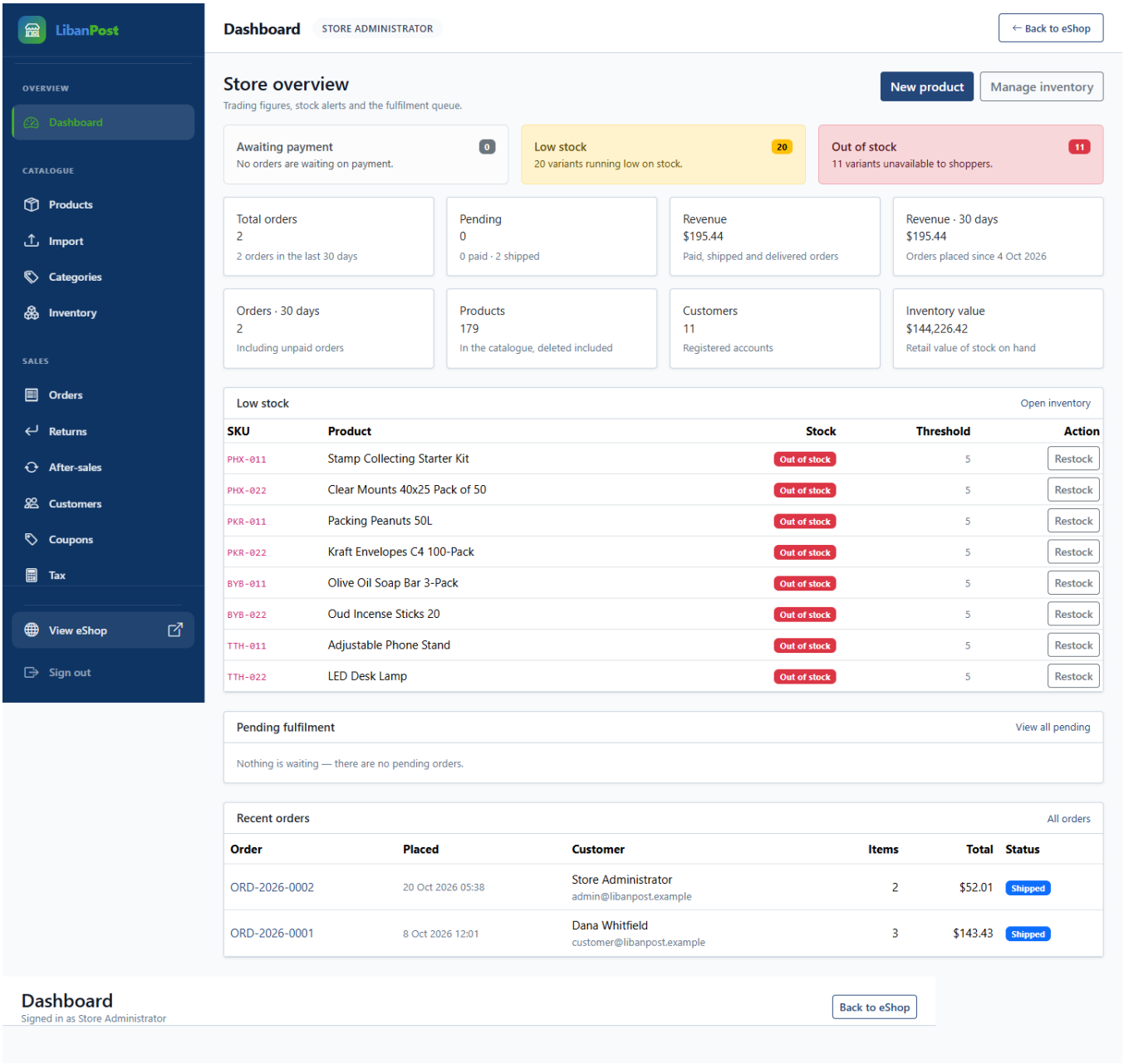
---

## Contents


- [The dashboard](#the-dashboard)
- [Products](#products)
- [Bulk import](#bulk-import)
- [Categories](#categories)
- [Inventory](#inventory)
- [Orders](#orders)
- [Returns](#returns)
- [Customers](#customers)
- [Coupons](#coupons)
- [Tax classes](#tax-classes)
- [Shops](#shops)
- [Payouts](#payouts)
- [Marketing analytics](#marketing-analytics)
- [Reviews moderation](#reviews-moderation)
- [Email outbox](#email-outbox)
- [Audit trail](#audit-trail)
- [Exports](#exports)
- [Where the store's own log lives](#where-the-stores-own-log-lives)
- [Questions staff usually ask](#questions-staff-usually-ask)


---

## The dashboard

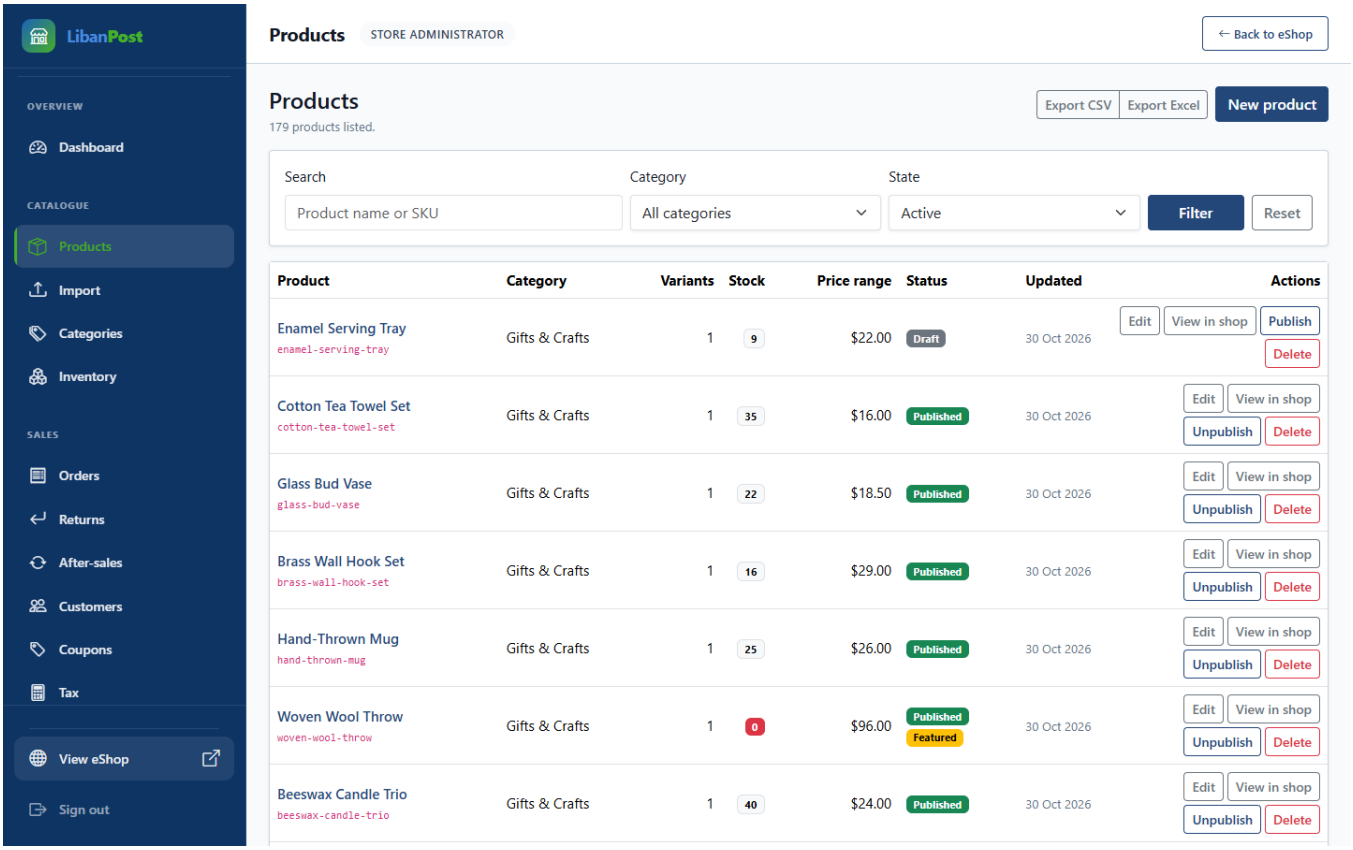



*Admin dashboard*

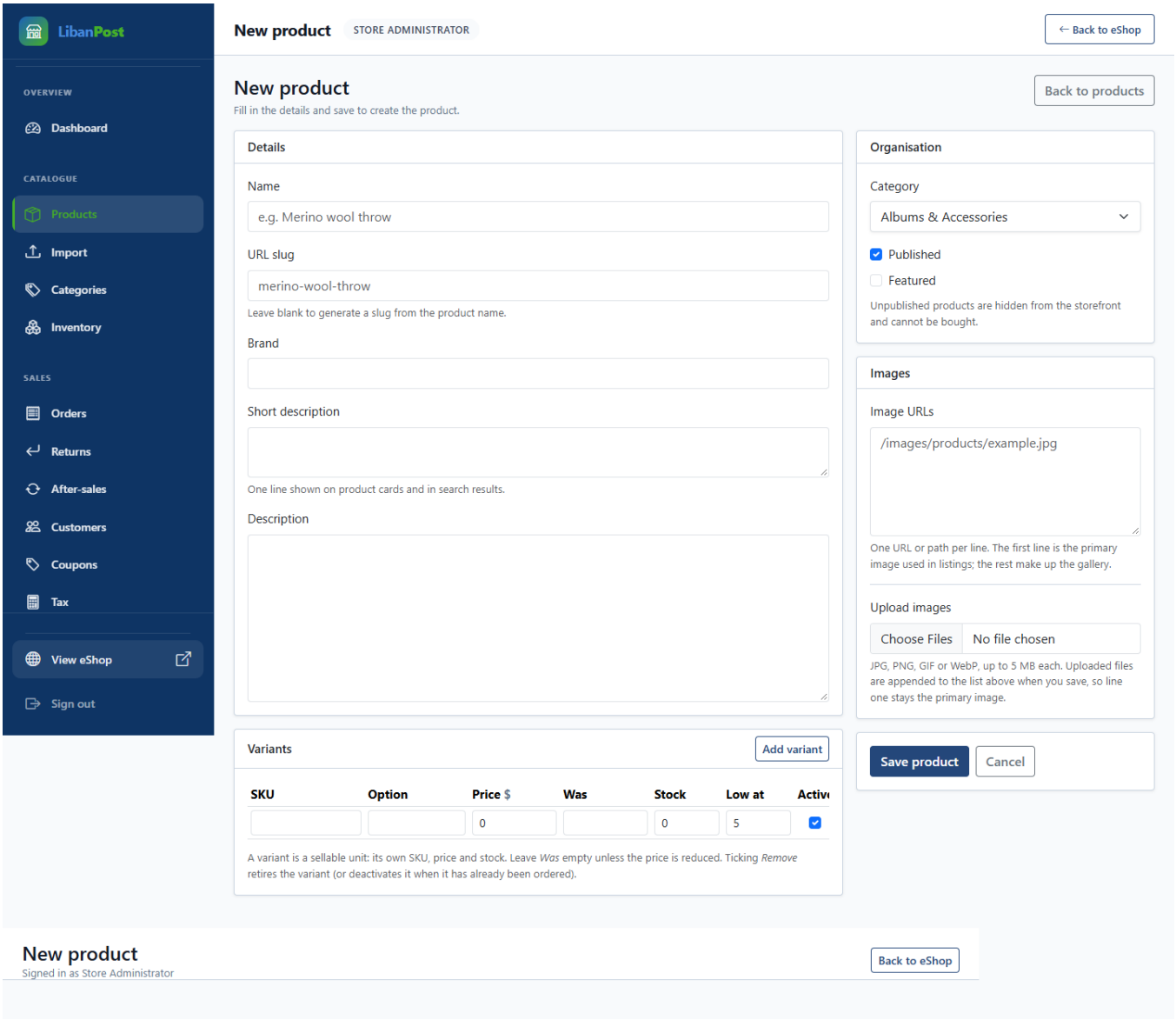
`/Admin/Dashboard` is the operational summary: revenue (lifetime and the last 30 days), order counts by state, the value of stock on hand, and stock alerts for anything at or below its low-stock threshold. It is the "is anything on fire?" page; the Marketing page is the "how are we doing?" page.

---

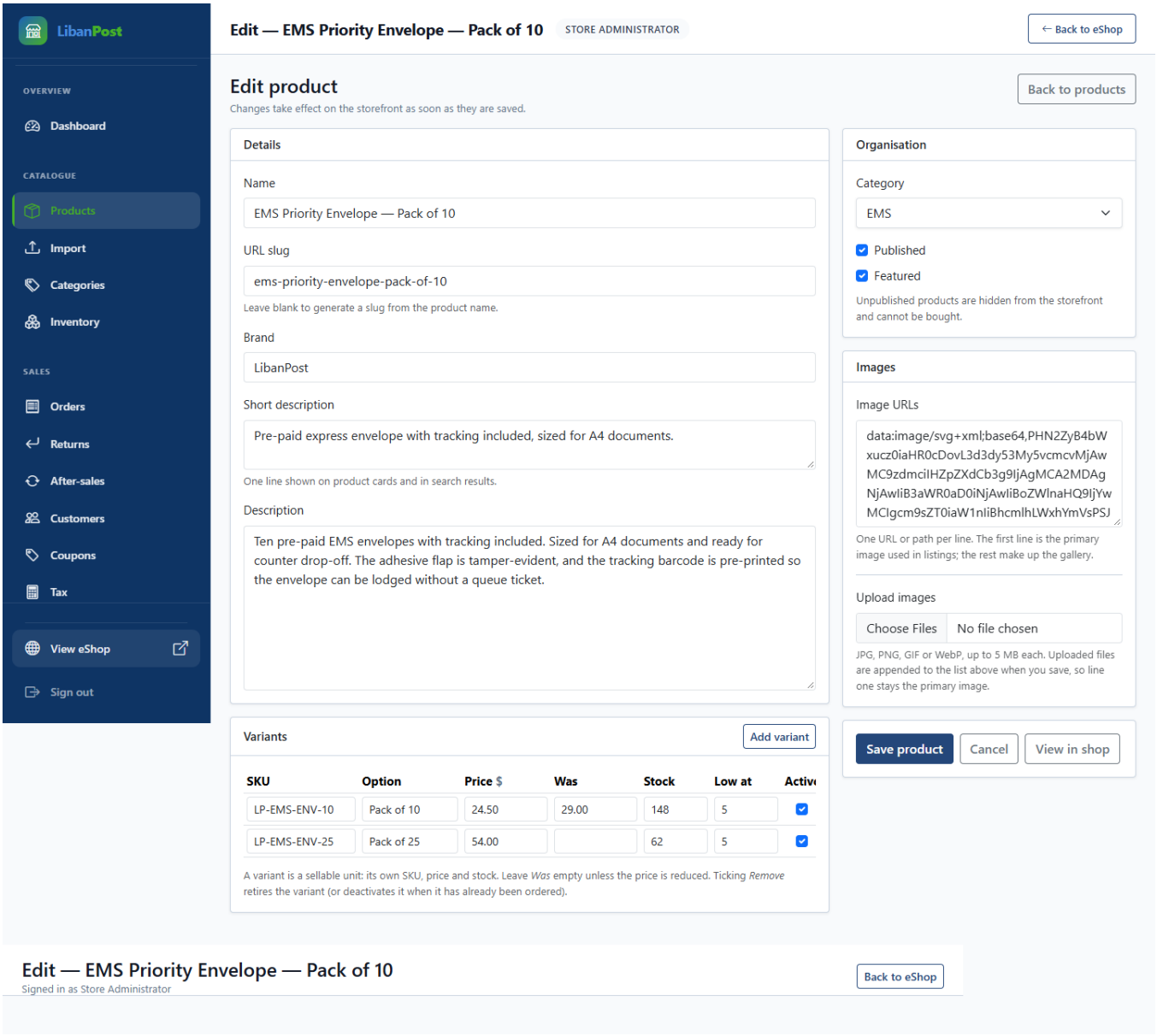
## Products




*Products list*




*Create product*




*Edit product*

`/Admin/Products` is the catalogue. It offers search, category and state filters (active, draft, out of stock, deleted), and the same editor a vendor uses — full variant matrix, image gallery that takes typed URLs *and* uploaded files, publish toggle, and soft delete with restore.

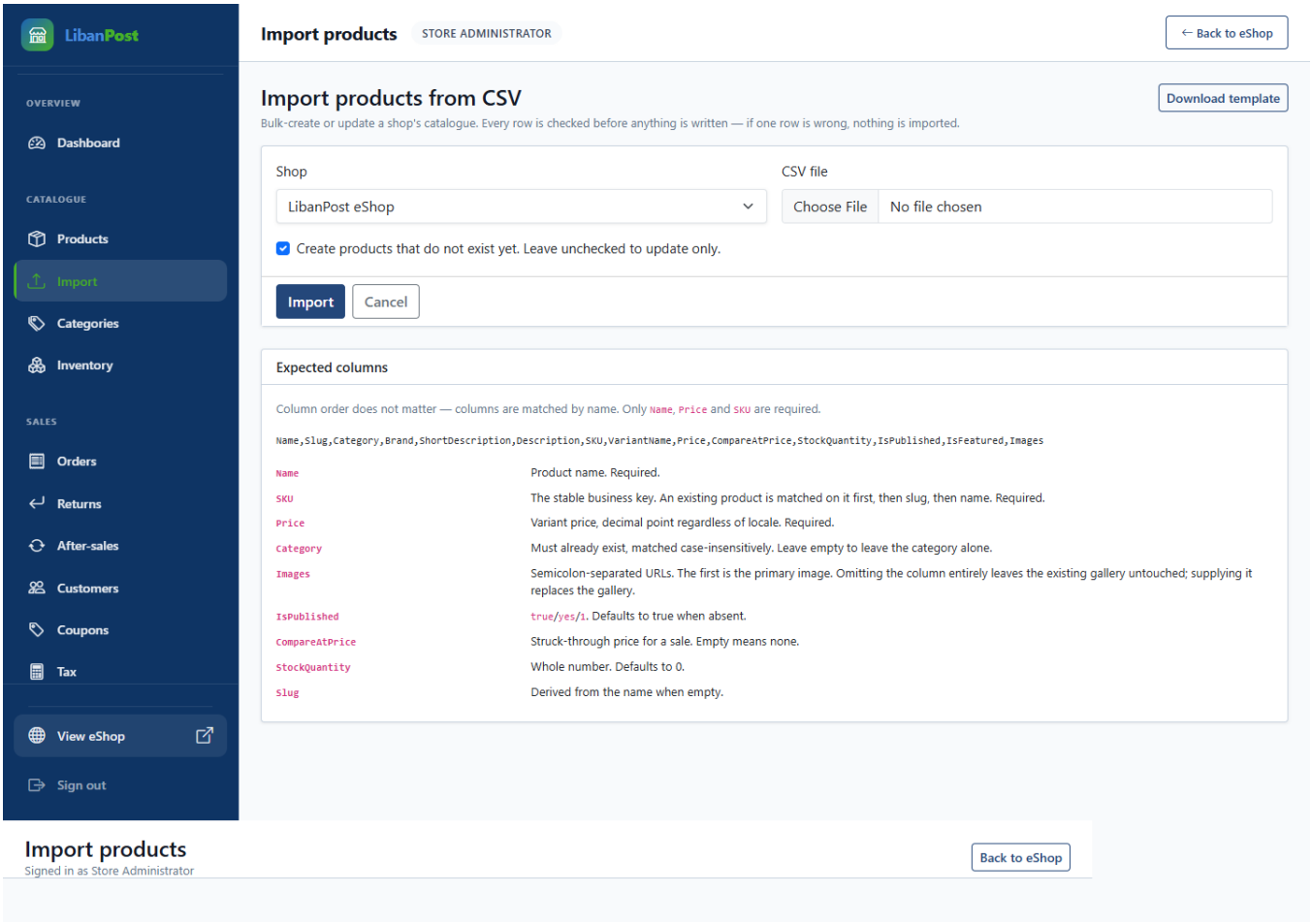
**Staff differ from vendors in one way that matters: staff see the whole catalogue and the soft-deleted rows**, so a withdrawn product can be found and restored rather than being invisible forever.

Because orders snapshot the product name, variant name, SKU and unit price onto each order line, **editing or deleting a product never rewrites the history of an order that already contains it.**

Use **Export CSV** / **Export Excel** to download exactly what the filters currently select.

---

## Bulk import




*Bulk import*

`/Admin/Import` loads a CSV of products into a chosen shop. It is all-or-nothing, validates every row before writing anything, and reports failures by line number. Column order does not matter; headers are matched by name. `Name`, `Price` and `SKU` are required.

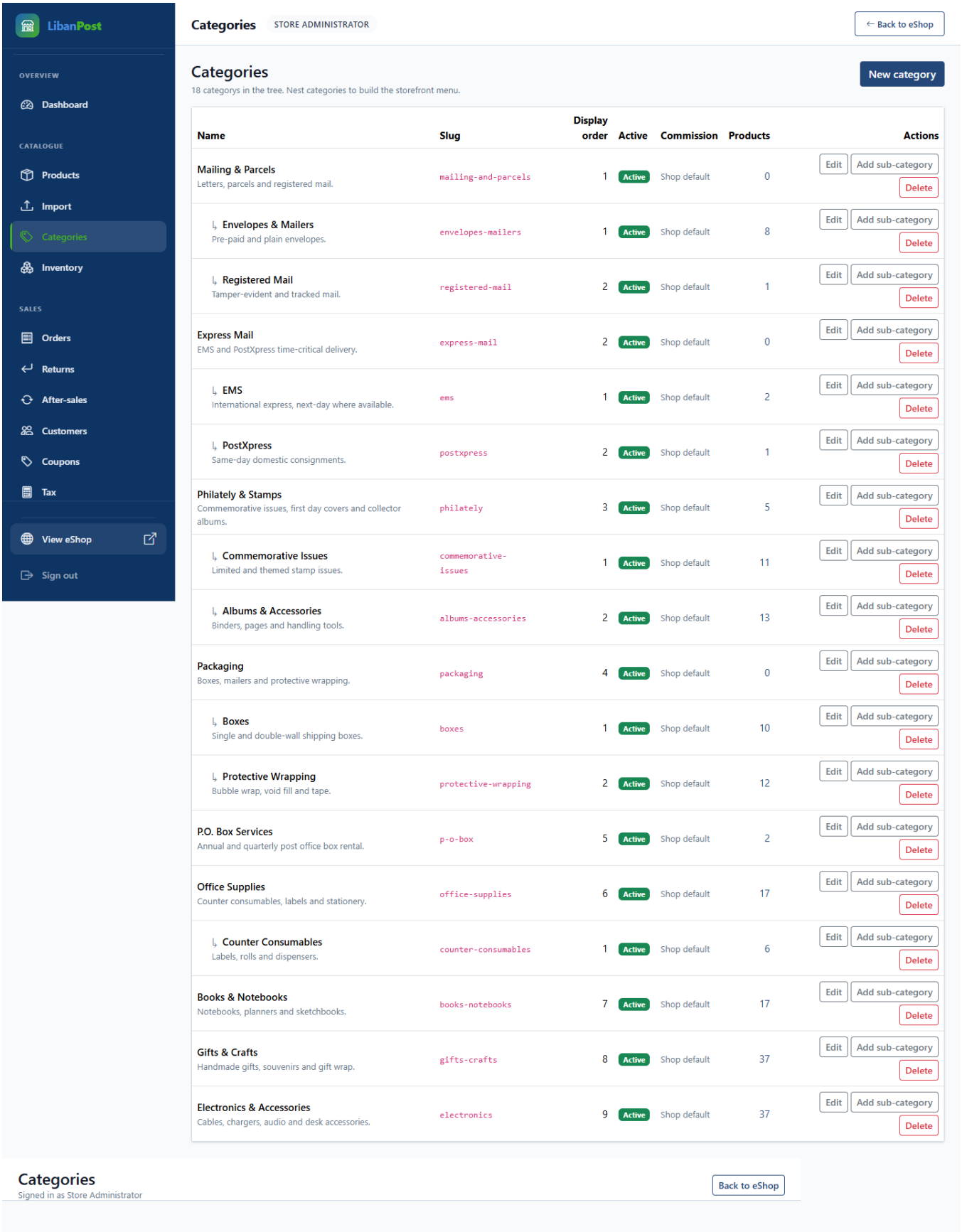
The importer is the exact inverse of the store's CSV export, so you can **export products, edit them in a spreadsheet, and import the file straight back**. Key behaviours:

- Matching is by **SKU** → **slug** → **name**, in that order.
- `Category` must already exist.
- Omitting the `Images` column **leaves the existing gallery untouched**; supplying it replaces the gallery.
- Turning **off** "create products that do not exist yet" makes it an update-only import.

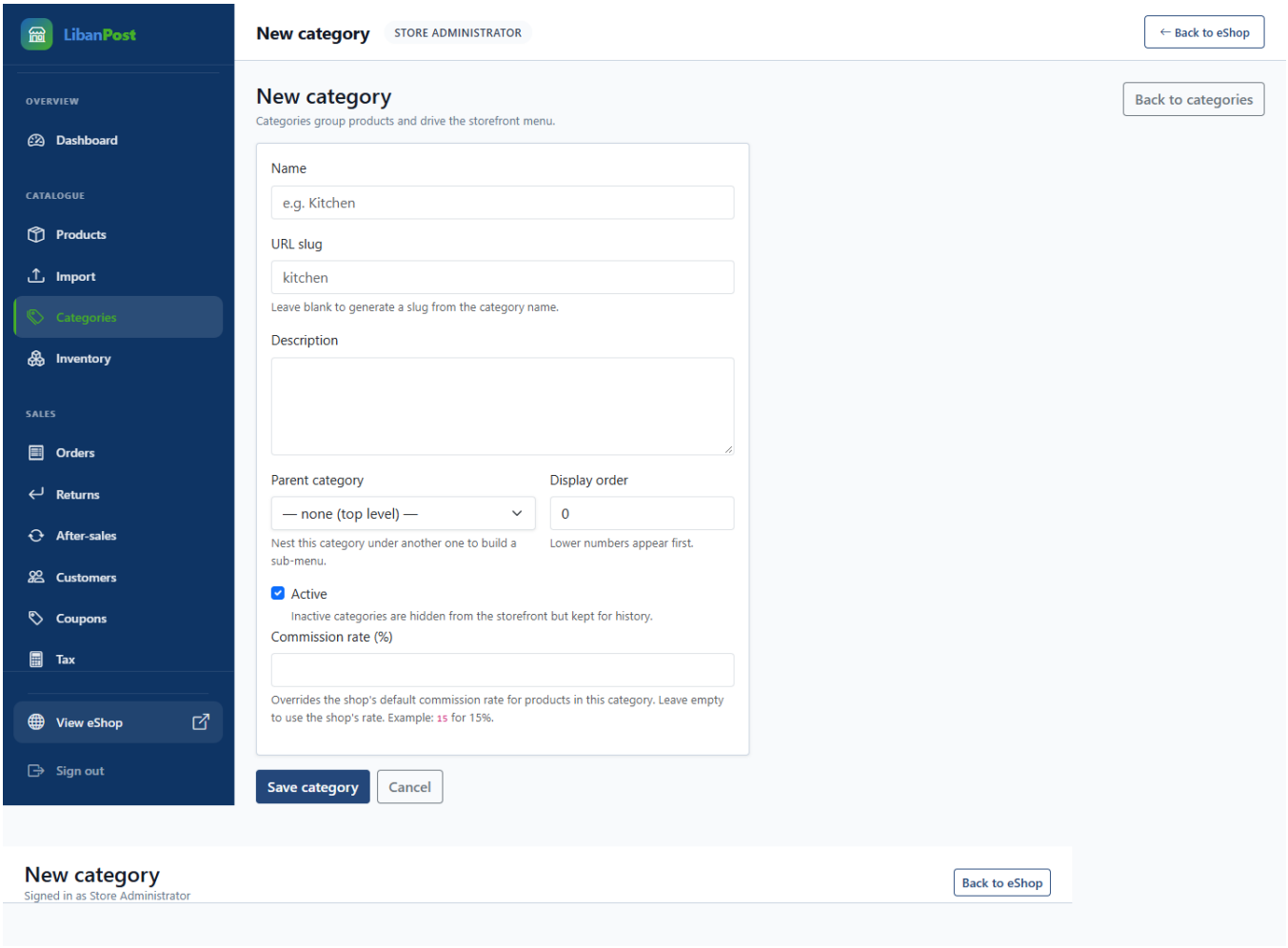
Download the template from the same page for a correct starting header. See the vendor guide for the full column reference.

---

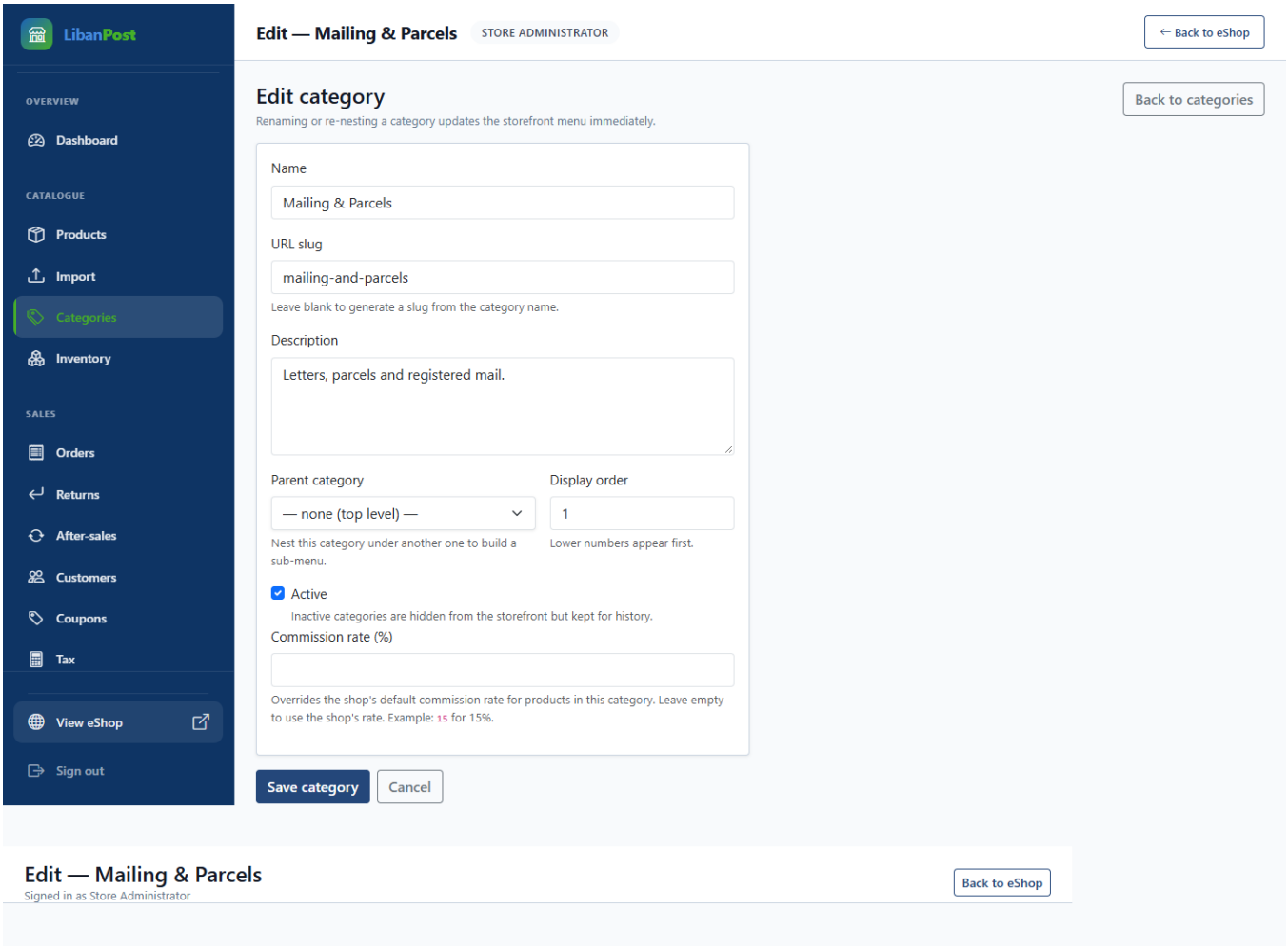
## Categories


LibanPost
Categories
STORE ADMINISTRATOR
← Back to eShop
OVERVIEW
Dashboard
CATALOGUE
Products
Import
Categories
Inventory
SALES
Orders
Returns
After-sales
Customers
Coupons
Tax
View eShop
Sign out
Categories
18 categorys in the tree. Nest categories to build the storefront menu.
New category
Name
Slug
Display order
Active
Commission
Products
Actions
Mailing & Parcels
Letters, parcels and registered mail.
mailing-and-parcels
1
Active
Shop default
0
Edit
Add sub-category
Delete
Envelopes & Mailers
Pre-paid and plain envelopes.
envelopes-mailers
1
Active
Shop default
8
Edit
Add sub-category
Delete
Registered Mail
Tamper-evident and tracked mail.
registered-mail
2
Active
Shop default
1
Edit
Add sub-category
Delete
Express Mail
EMS and PostXpress time-critical delivery.
express-mail
2
Active
Shop default
0
Edit
Add sub-category
Delete
EMS
International express, next-day where available.
ems
1
Active
Shop default
2
Edit
Add sub-category
Delete
PostXpress
Same-day domestic consignments.
postxpress
2
Active
Shop default
1
Edit
Add sub-category
Delete
Philately & Stamps
Commemorative issues, first day covers and collector albums.
philately
3
Active
Shop default
5
Edit
Add sub-category
Delete
Commemorative Issues
Limited and themed stamp issues.
commemorative-issues
1
Active
Shop default
11
Edit
Add sub-category
Delete
Albums & Accessories
Binders, pages and handling tools.
albums-accessories
2
Active
Shop default
13
Edit
Add sub-category
Delete
Packaging
Boxes, mailers and protective wrapping.
packaging
4
Active
Shop default
0
Edit
Add sub-category
Delete
Boxes
Single and double-wall shipping boxes.
boxes
1
Active
Shop default
10
Edit
Add sub-category
Delete
Protective Wrapping
Bubble wrap, void fill and tape.
protective-wrapping
2
Active
Shop default
12
Edit
Add sub-category
Delete
P.O. Box Services
Annual and quarterly post office box rental.
p-o-box
5
Active
Shop default
2
Edit
Add sub-category
Delete
Office Supplies
Counter consumables, labels and stationery.
office-supplies
6
Active
Shop default
17
Edit
Add sub-category
Delete
Counter Consumables
Labels, rolls and dispensers.
counter-consumables
1
Active
Shop default
6
Edit
Add sub-category
Delete
Books & Notebooks
Notebooks, planners and sketchbooks.
books-notebooks
7
Active
Shop default
17
Edit
Add sub-category
Delete
Gifts & Crafts
Handmade gifts, souvenirs and gift wrap.
gifts-crafts
8
Active
Shop default
37
Edit
Add sub-category
Delete
Electronics & Accessories
Cables, chargers, audio and desk accessories.
electronics
9
Active
Shop default
37
Edit
Add sub-category
Delete
Categories
Signed in as Store Administrator
Back to eShop

*Categories*




*Create category*

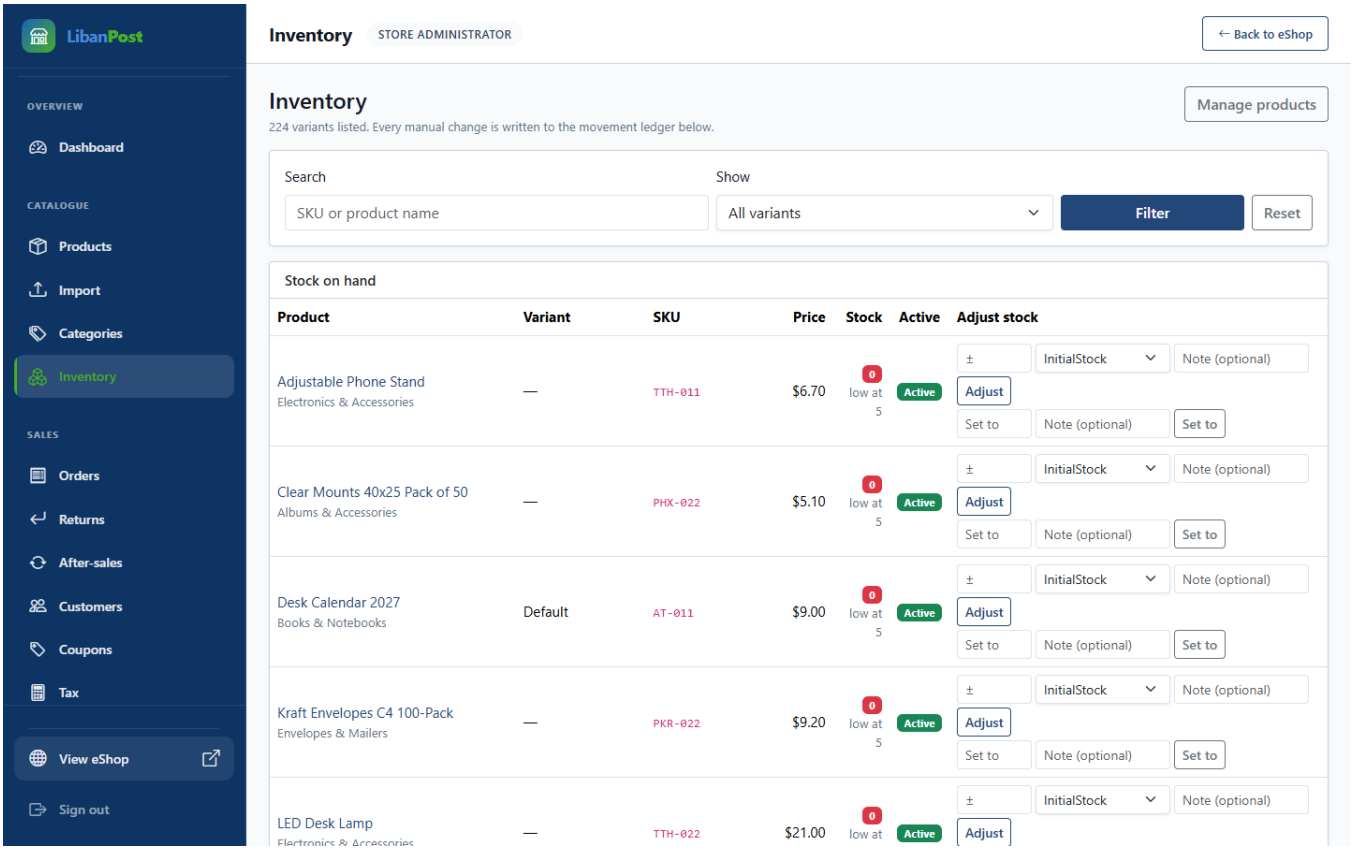



*Edit category*

`/Admin/Categories` manages the catalogue's tree with indentation and per-category product counts. Categories are global to the marketplace, so they are shared across every shop. The form guards against cycles — you cannot make a category its own ancestor.

---

## Inventory



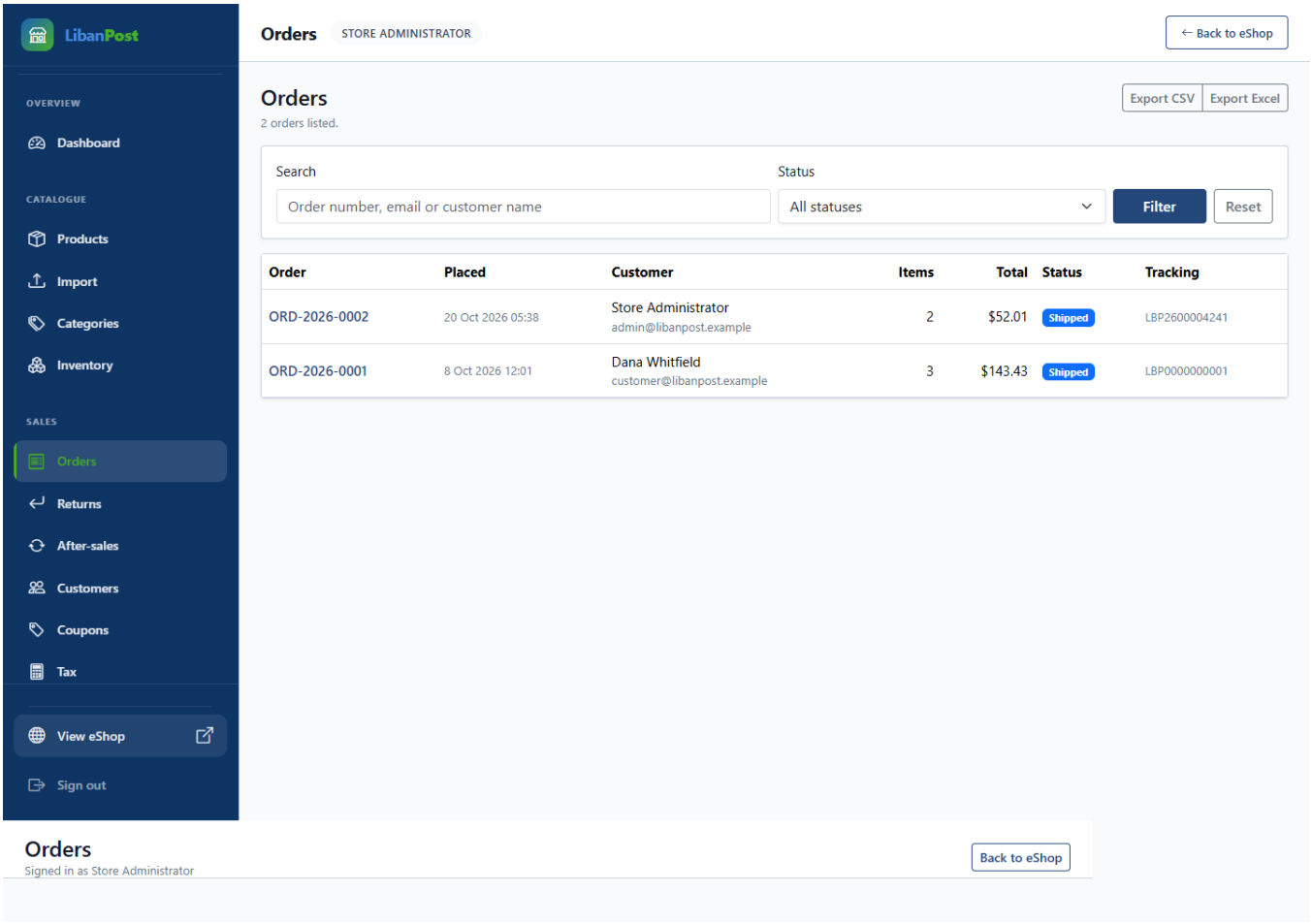

*Inventory*

`/Admin/Inventory` shows stock levels across products and lets you adjust them. Every change is written to an **append-only stock-movement ledger**, so any movement can be explained after the fact: who changed what, by how much, and why.

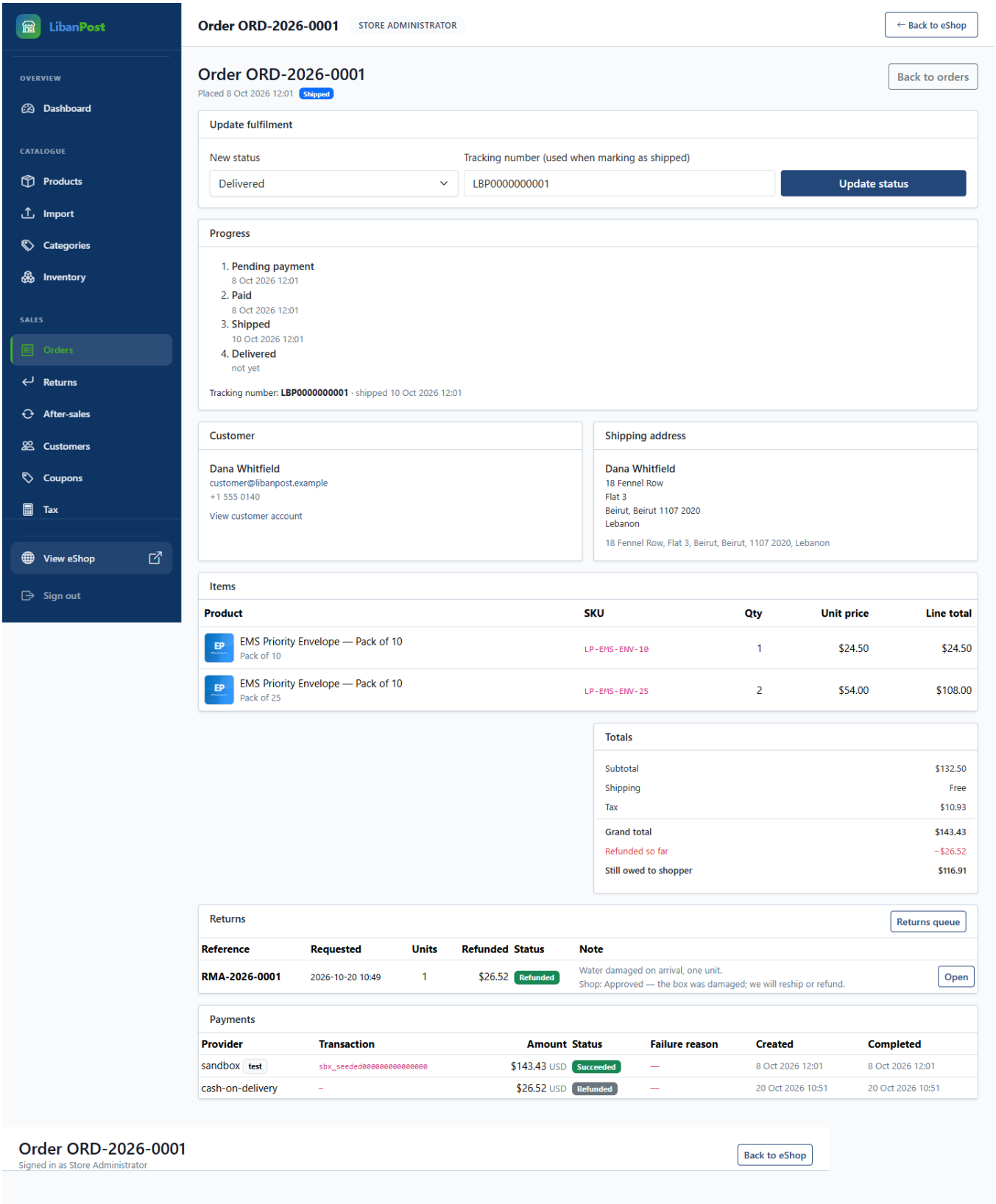
Stock is committed under **optimistic concurrency**. If two customers check out the last item at the same instant, one of them gets a conflict rather than both being sold something that does not exist; the order service retries with a fresh basket and, if it still cannot be filled, tells the customer plainly.

---

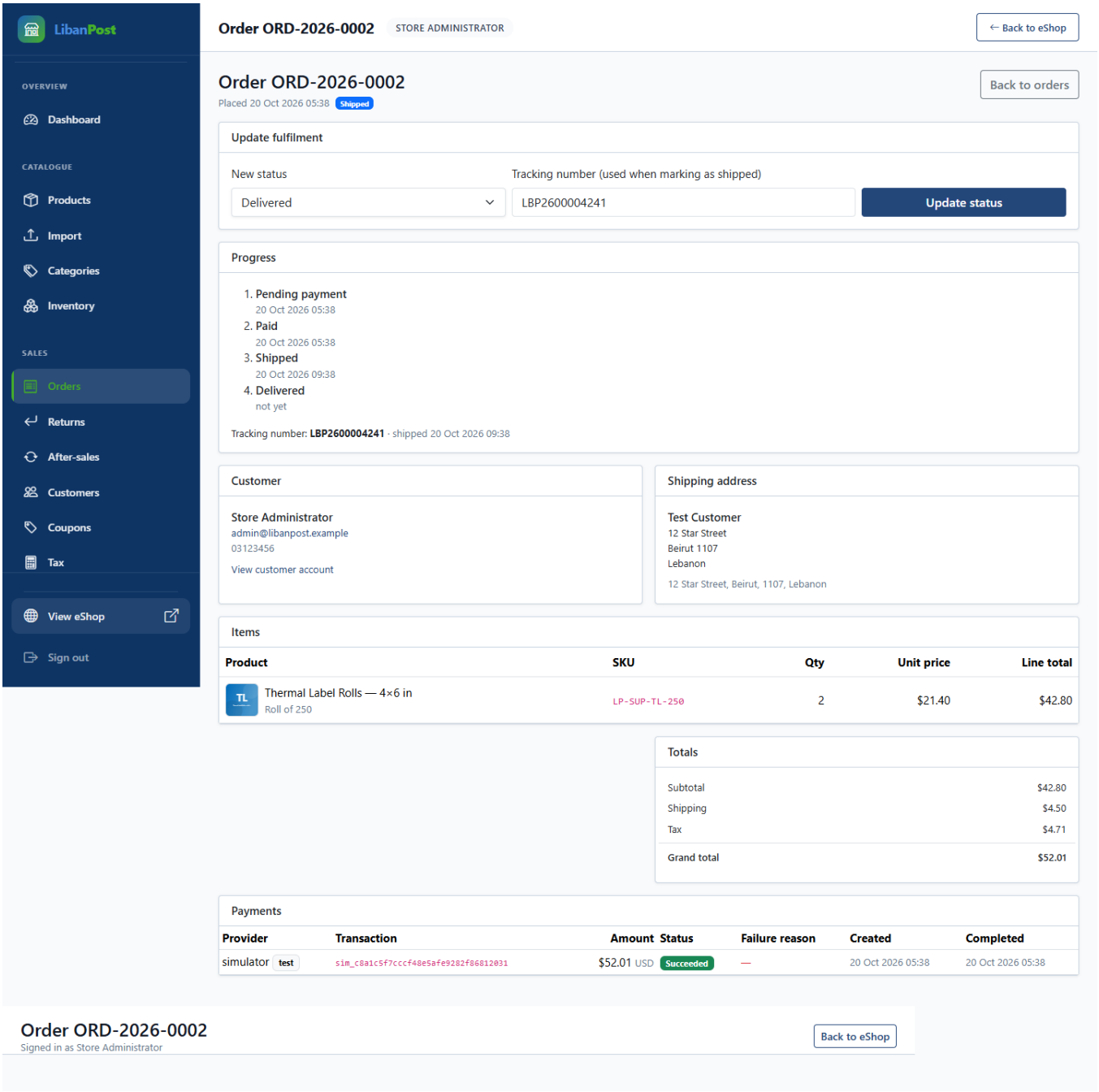
## Orders


LibanPost
Orders
STORE ADMINISTRATOR
← Back to eShop
OVERVIEW
Dashboard
CATALOGUE
Products
Import
Categories
Inventory
SALES
Orders
Returns
After-sales
Customers
Coupons
Tax
View eShop
Sign out
Orders
2 orders listed.
Export CSV
Export Excel
Search
Order number, email or customer name
Status
All statuses
Filter
Reset
Order
Placed
Customer
Items
Total
Status
Tracking
ORD-2026-0002
20 Oct 2026 05:38
Store Administrator
admin@libanpost.example
2
$52.01
Shipped
LBP2600004241
ORD-2026-0001
8 Oct 2026 12:01
Dana Whitfield
customer@libanpost.example
3
$143.43
Shipped
LBP0000000001
Orders
Signed in as Store Administrator
Back to eShop

*Orders list*




*Order detail*




*Order detail (second)*

`/Admin/Orders` is the fulfilment queue.

- **Filter** by status and search by customer or order number.
- **Open an order** for its detail: lines, addresses, payment, collection status and tracking.
- **Change status** with `UpdateStatus`, subject to the order state machine — only legal transitions are permitted.

What the transitions actually do:

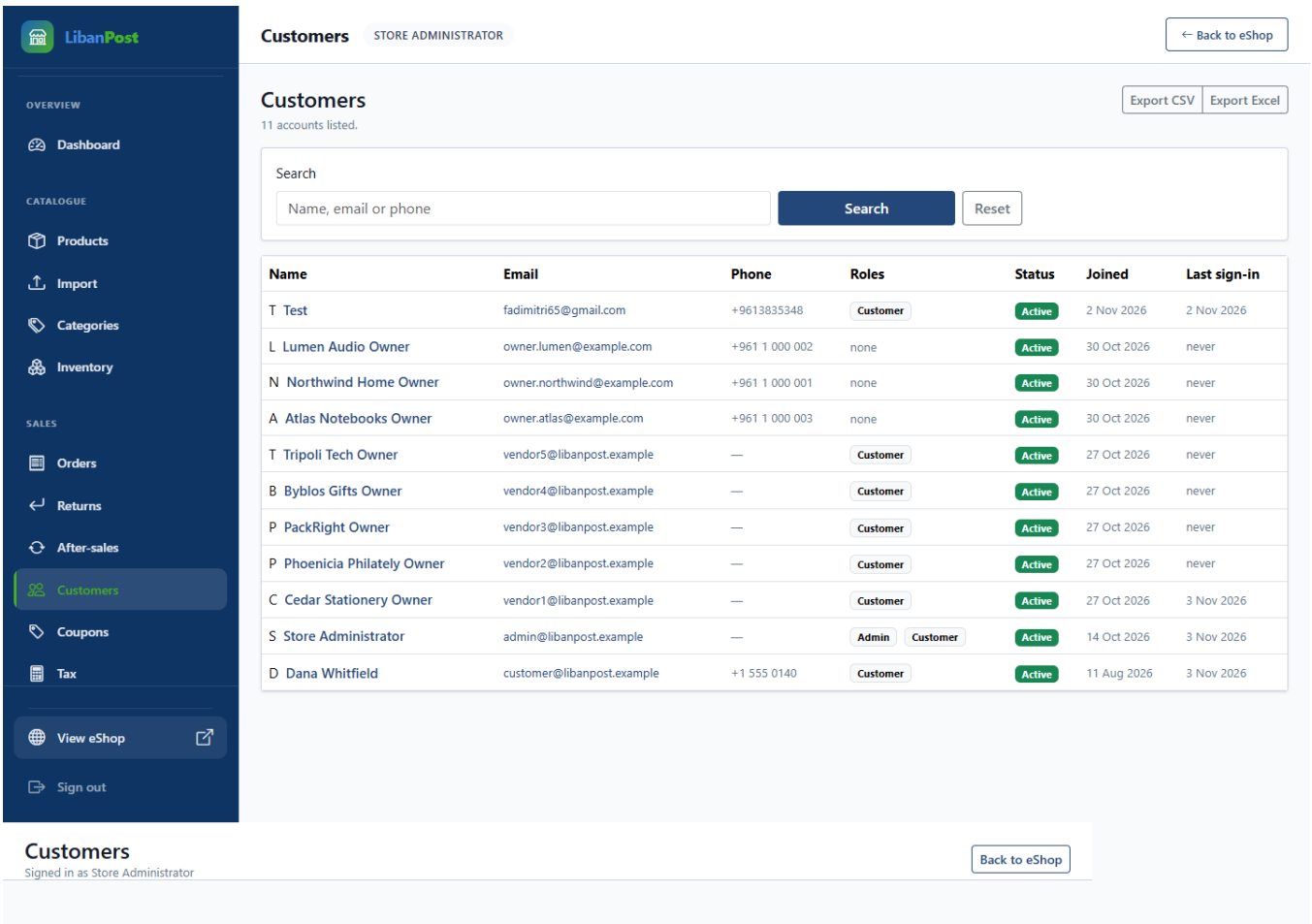
| Transition | Effect |
|---|---|
| → **Paid** | Credits the vendor ledger (money has been collected). |
| → **Shipped** | Queues a shipping notification email, with tracking. |
| → **Cancelled** | **Restores stock** and reverses the vendor's earning. |
| → **Refunded** | Calls the payment gateway; reverses the vendor's earning. **Stock is not restored** (the goods left). |

Order-by-order refunds are whole-order actions. Per-line refunds with goods coming back run through the returns desk (see [Returns](#returns)); a full-return refund there also moves the order to *Refunded* and flips the cash-on-delivery collection to *refunded*.

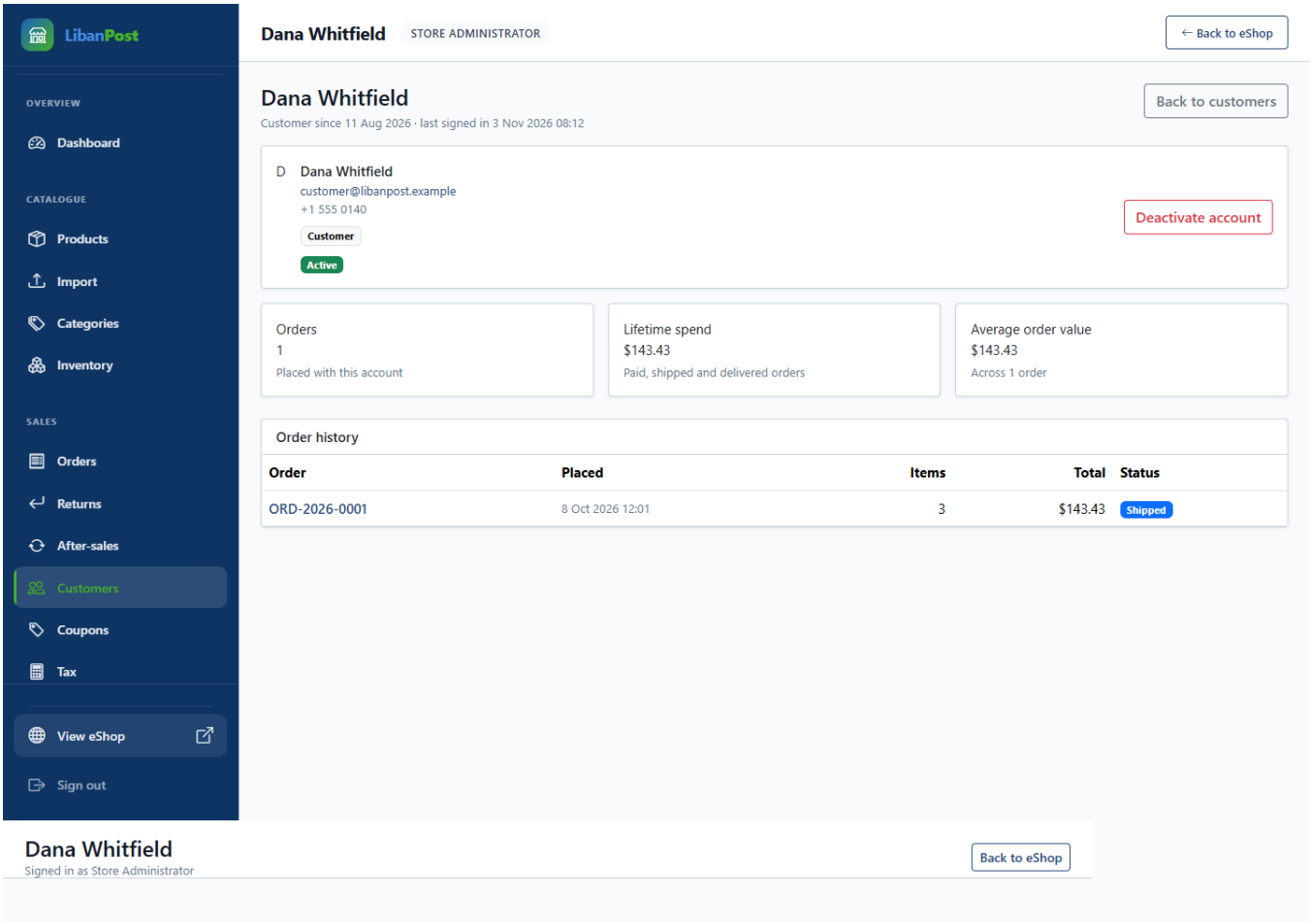
Export the current filter selection as CSV or Excel.

---

## Customers




*Customers*

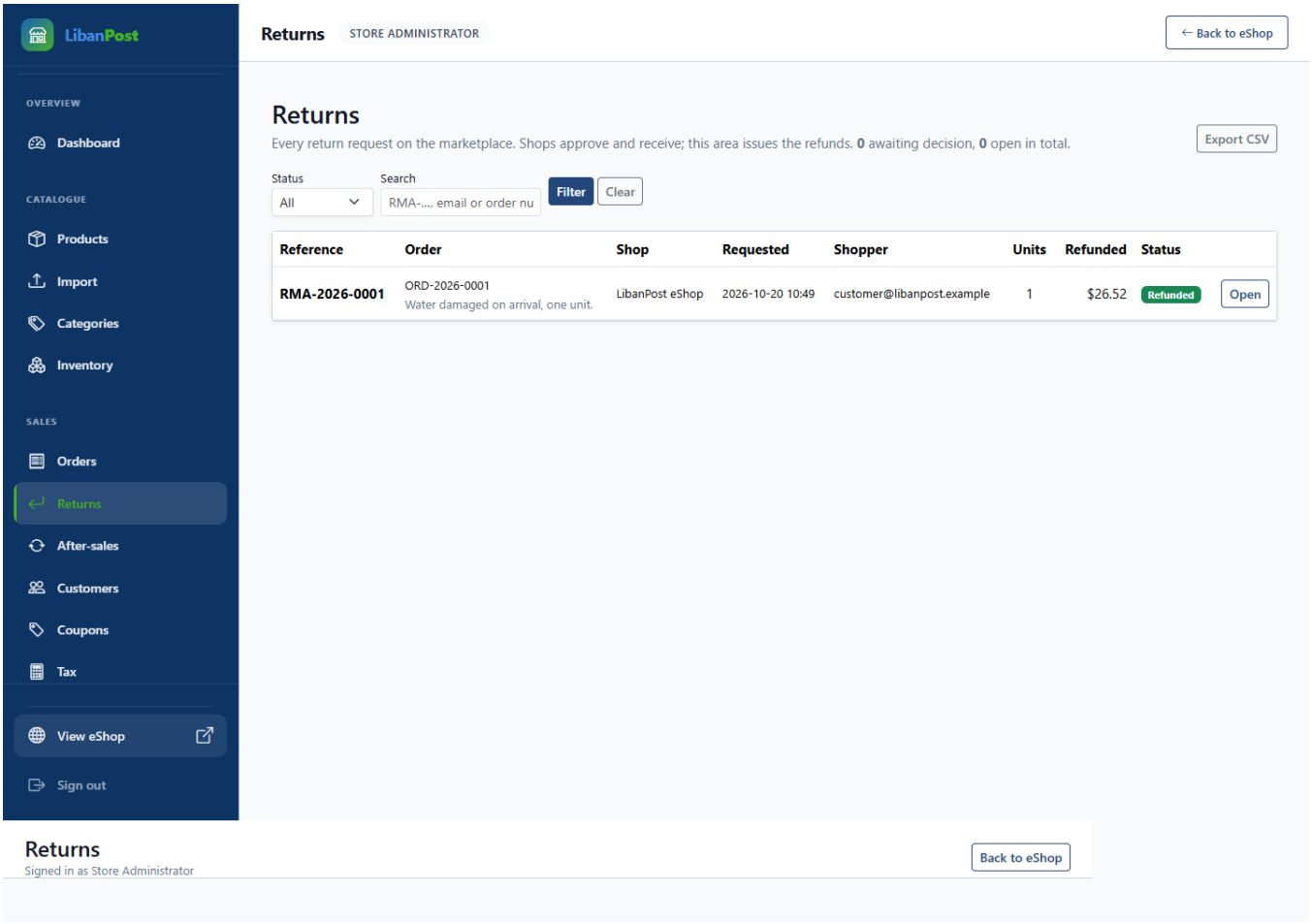



*Customer detail*

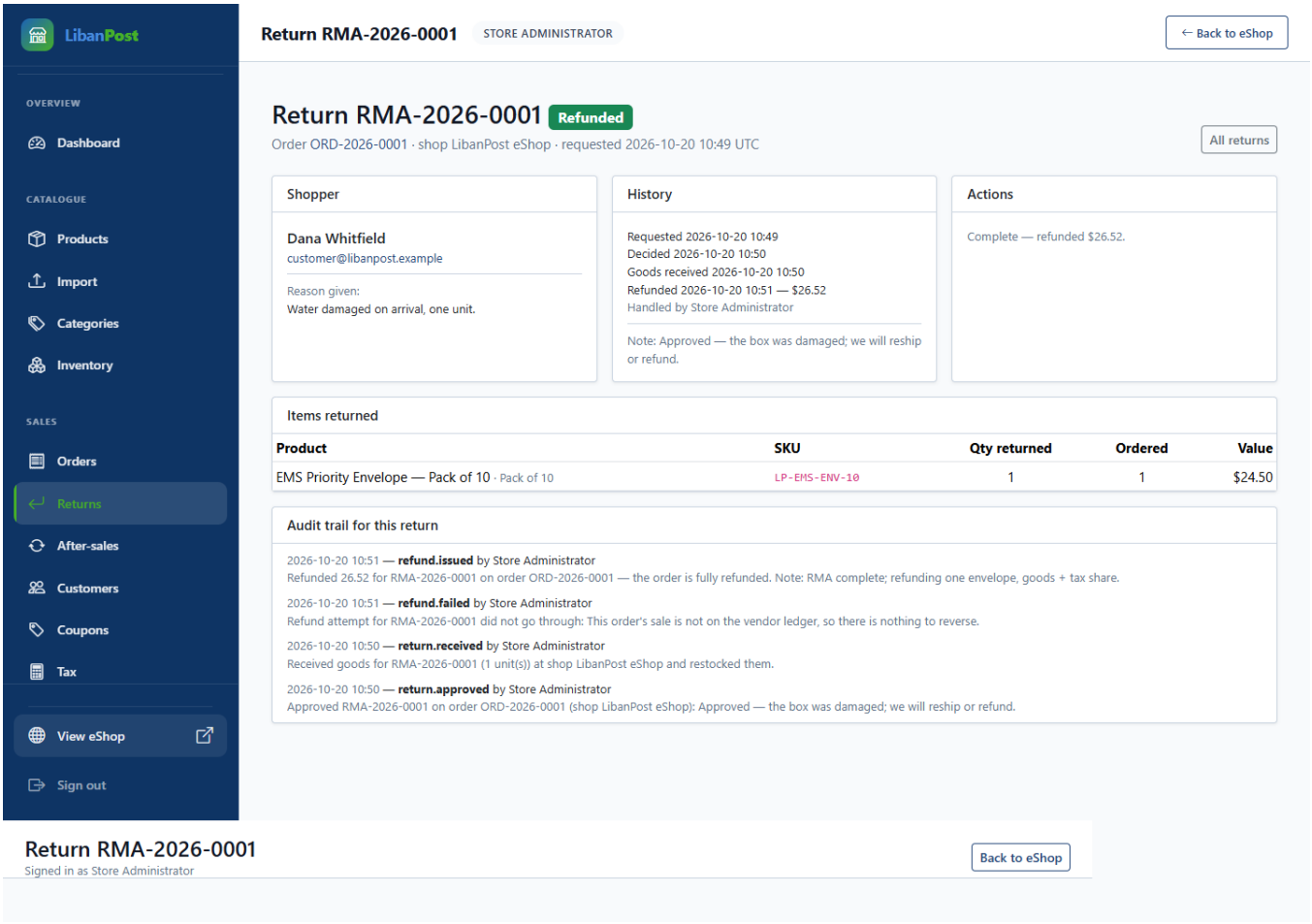
`/Admin/Customers` searches accounts and shows a detail page with **lifetime spend**. Accounts can be **activated or deactivated** — a deactivated account cannot sign in, but its order history stays intact. Deactivating is preferred over deleting, for exactly that reason.

---

## Returns




*Returns queue*




*Return detail*

`/Admin/Returns` is the marketplace's RMA desk. The marketplace's job here is money and covers: the shops decide their own returns day to day, but staff can act for them and are the only role that issues the actual refund.

- **The queue** — filter by status, or search by RMA reference, shopper email or order number.

Counters above the table show how many requests sit at *requested*, *approved* and *received* — the unfinished work. Export the current view as CSV or Excel.

- **Deciding** — a return can be approved or rejected from here, with a note, exactly as the vendor

would; handy when a shop is slow to answer and a shopper is waiting. Staff decisions are what the audit trail credits to the *administrator's* name, not the shop's.

- **Receiving** — "goods received" restocks the returned units into the shop's listing stock (or the

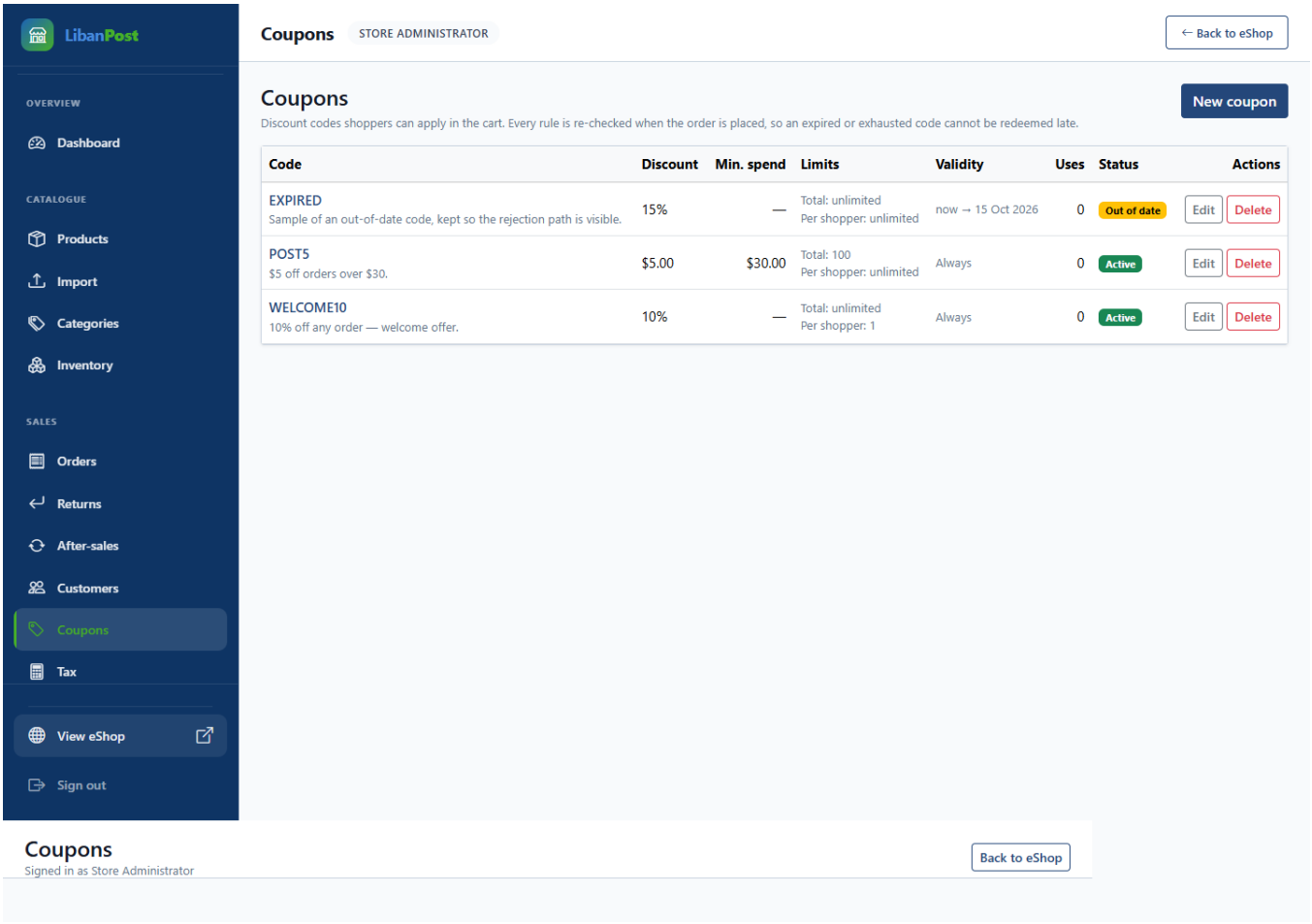
shared variant pool if the listing was withdrawn) and writes a return-stamped stock movement.

- **Refunding** — press **Issue refund** on a *received* return. The amount is computed, not typed:

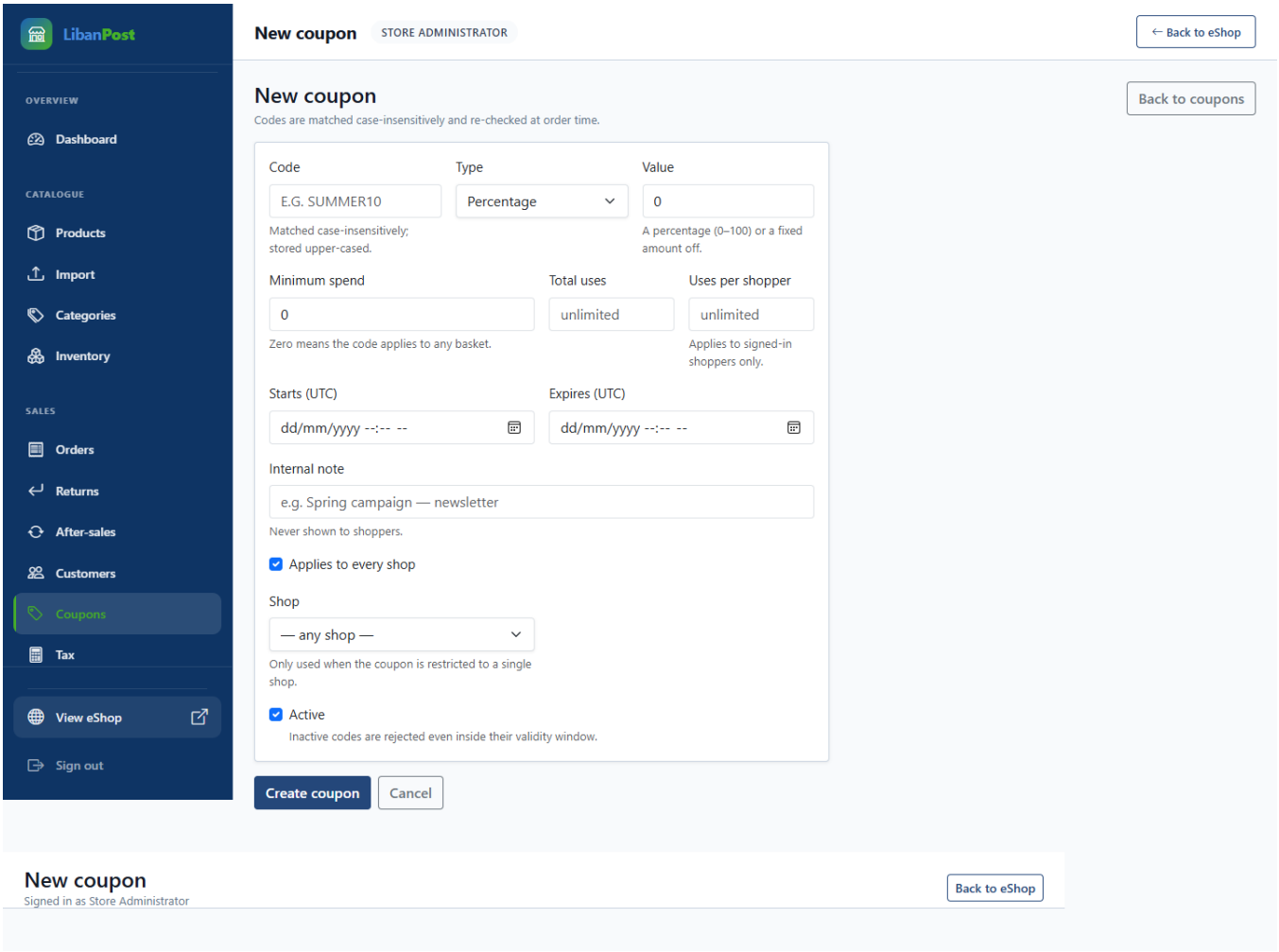
the goods actually returned (with their share of any order discount) plus the proportionate tax, and shipping only when the whole order comes back. It is capped by whatever the ledger still owes the shop for that sale and by `GrandTotal − RefundedTotal`, so it can never exceed what the order received. A card refund goes back against the original charge; a cash-on-delivery refund is a staff bookkeeping row.

A refund that cannot go through (no sale on the ledger to reverse, a state that forbids the step) is refused with a plain reason and **recorded on the audit trail as a failed attempt** — the trail should explain the retries, not hide them. The order's page keeps *Refunded so far* and *Still owed to shopper* totals, and the vendor sees the same return from their side at `/Vendor/Returns`.

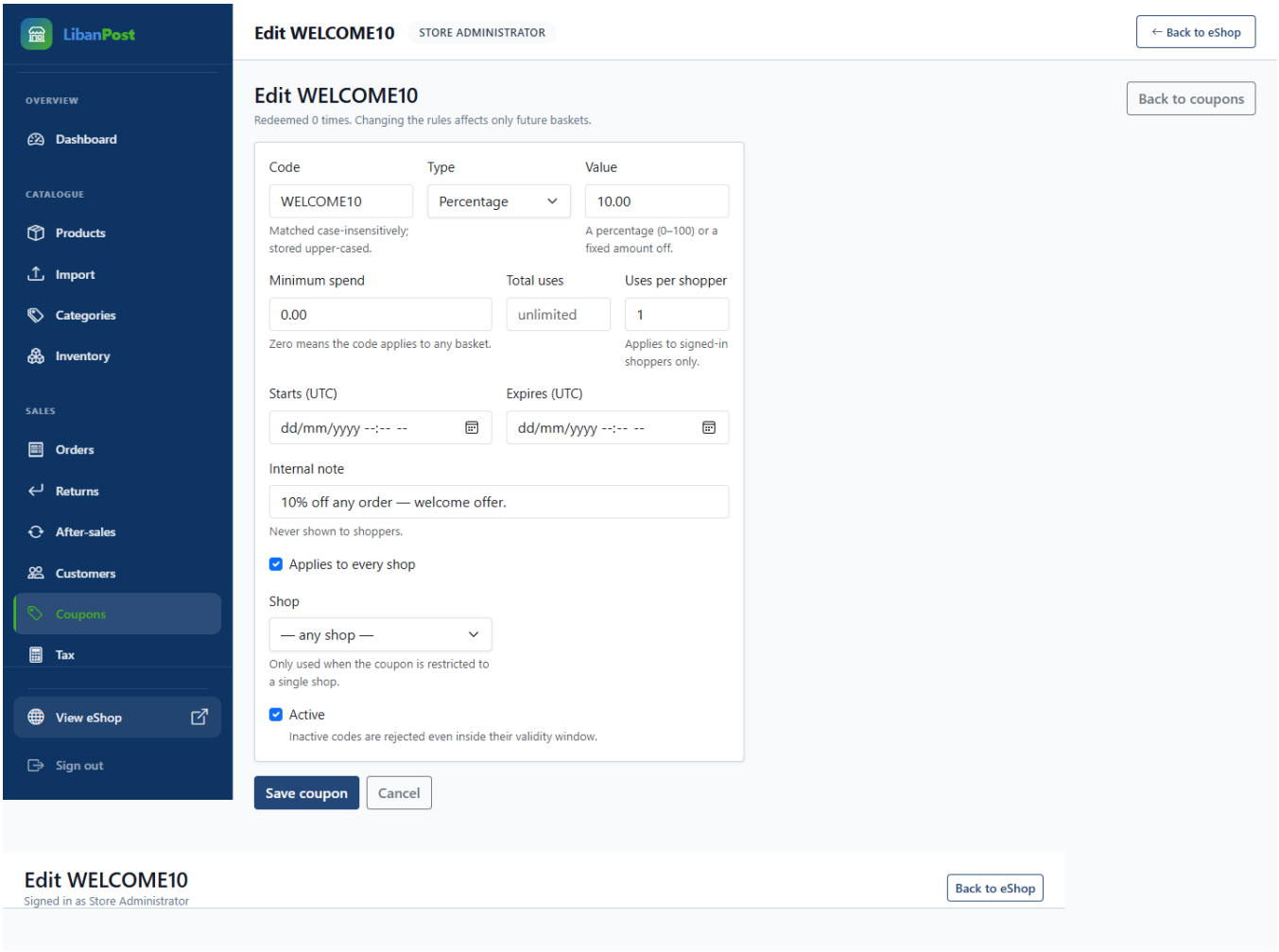
# Coupons




*Coupons*




*Create coupon*




*Edit coupon*

`/Admin/Coupons` creates and edits discount codes:

- **Type** — percentage or fixed amount.
- **Minimum subtotal**, and a **validity window** (`StartsUtc` / `ExpiresUtc`).
- `MaxUses` (global) and `MaxUsesPerUser` (per shopper).
- Optional restriction to **one shop**.
- An **activate toggle**.

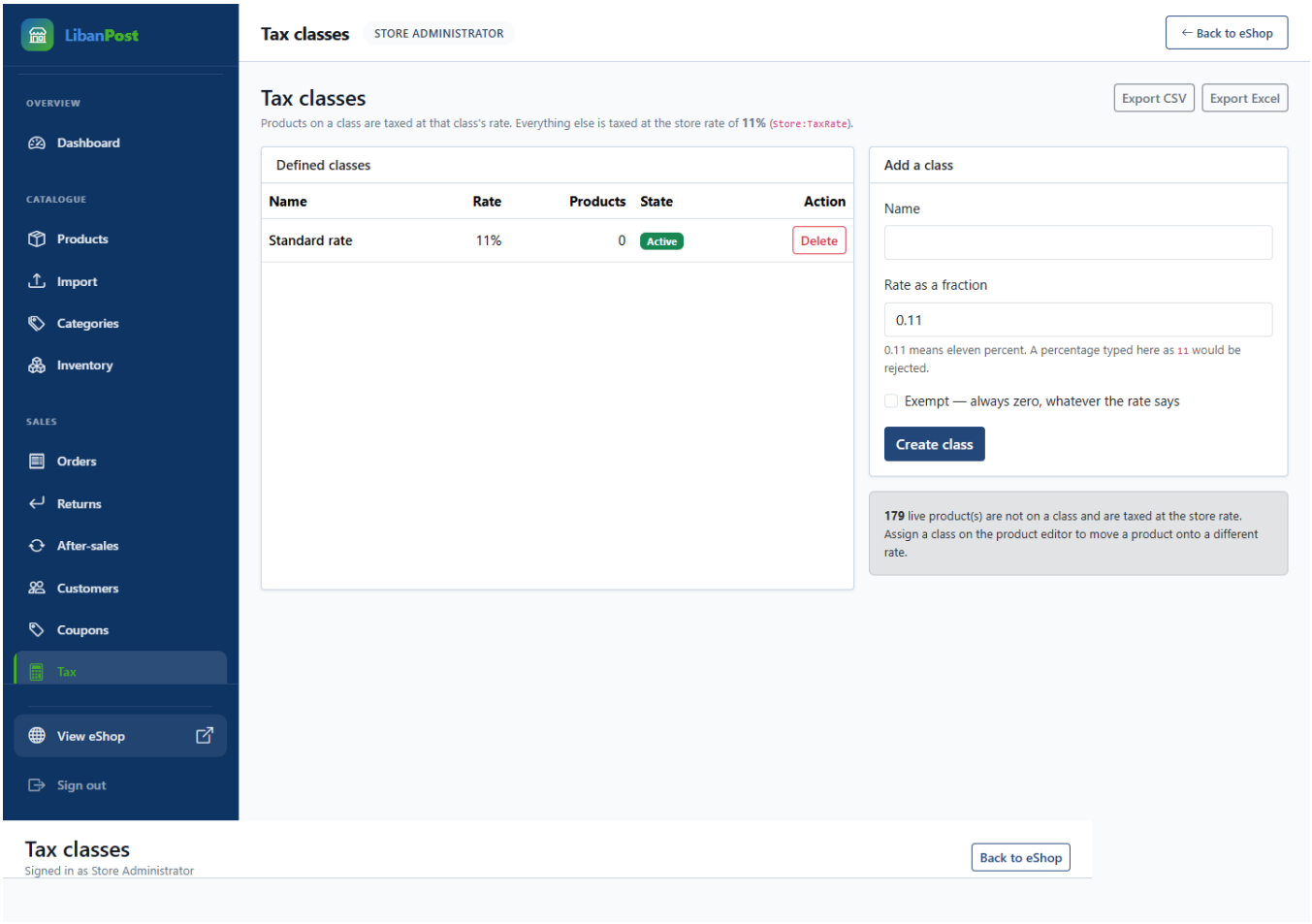
Validation refuses nonsense at the form: a percentage over 100, an out-of-range fixed amount, a start after the expiry, and a shop-scoped code with no shop are all rejected.

**A code that has already been redeemed is deactivated rather than deleted**, so order history that references it stays honest. Every redemption is recorded as a `CouponUsage` row holding the user and the order — which is how the per-shopper cap is enforced without trusting the browser. Guests are held only to the global limit.

Note the deliberate limits: coupons **do not stack**, and they are **not line-item aware** — one code per order, applied to the goods subtotal.

---

## Tax classes




*Tax classes*

`/Admin/Tax` defines named tax rates. A product may point at one; a product with no class is taxed at the store-wide rate (`Store:TaxRate`).

- Rates are entered as a **fraction**: `0.11` means eleven percent. The form refuses anything outside 0–1, so nobody can type `11` and charge 1100%.
- A class can be marked **exempt** — always zero, whatever the rate says — for VAT-registered buyers and resale certificates.
- **Deleting is guarded.** If products still reference a class, it is **deactivated rather than deleted**, because deleting it would silently move those products onto the store rate — a real change to what a shopper pays, made without anyone deciding to make it.
- The page shows how many live products are on each class and how many are on the store default.

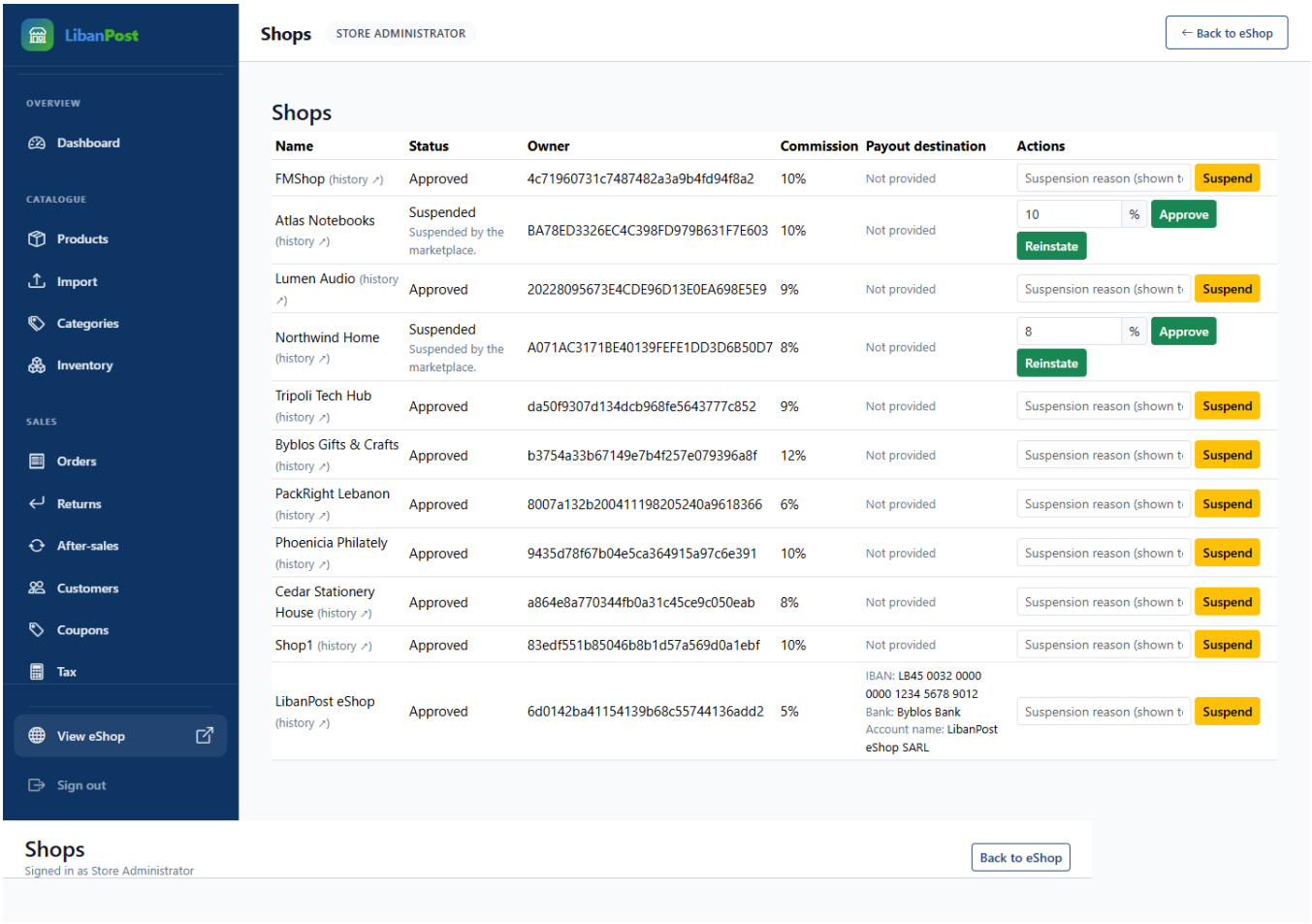
Tax is applied **per line, from each line's own class**, with any order-level discount apportioned across the lines first. That means a basket holding both standard-rated and zero-rated goods is taxed correctly, and the figure shown at checkout is the same one the order records.

**Products with no tax class behave exactly as they did before this feature existed** — the store rate applies, unchanged. A fresh store is seeded with one *Standard rate* class and leaves products unassigned, so introducing classes changes no existing order total.

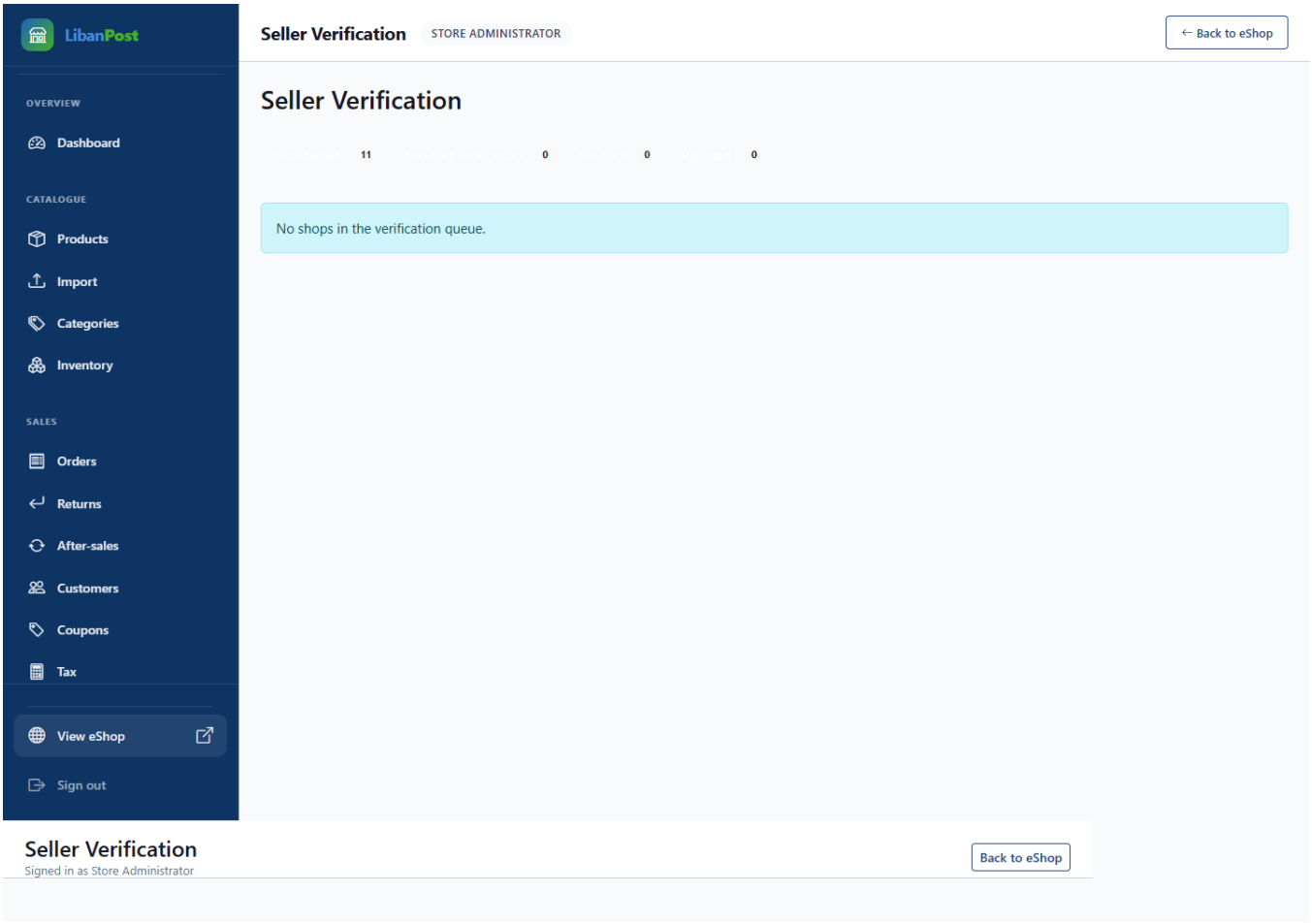
Export the class list as CSV or Excel.

---

## Shops


LibanPost
Shops
STORE ADMINISTRATOR
← Back to eShop
OVERVIEW
Dashboard
CATALOGUE
Products
Import
Categories
Inventory
SALES
Orders
Returns
After-sales
Customers
Coupons
Tax
View eShop
Sign out
Shops
Name
Status
Owner
Commission
Payout destination
Actions
FMShop (history ↗)
Approved
4c71960731c7487482a3a9b4fd94f8a2
10%
Not provided
Suspension reason (shown t
Suspend
Atlas Notebooks (history ↗)
Suspended
Suspended by the marketplace.
BA78ED3326EC4C398FD979B631F7E603
10%
Not provided
10
%
Approve
Reinstate
Lumen Audio (history ↗)
Approved
20228095673E4CDE96D13E0EA698E5E9
9%
Not provided
Suspension reason (shown t
Suspend
Northwind Home (history ↗)
Suspended
Suspended by the marketplace.
A071AC3171BE40139FEFE1DD3D6B50D7
8%
Not provided
8
%
Approve
Reinstate
Tripoli Tech Hub (history ↗)
Approved
da50f9307d134dcb968fe5643777c852
9%
Not provided
Suspension reason (shown t
Suspend
Byblos Gifts & Crafts (history ↗)
Approved
b3754a33b67149e7b4f257e079396a8f
12%
Not provided
Suspension reason (shown t
Suspend
PackRight Lebanon (history ↗)
Approved
8007a132b20041119820524 0a9618366
6%
Not provided
Suspension reason (shown t
Suspend
Phoenicia Philately (history ↗)
Approved
9435d78f67b04e5ca364915a97c6e391
10%
Not provided
Suspension reason (shown t
Suspend
Cedar Stationery House (history ↗)
Approved
a864e8a770344fb0a31c45ce9c050eab
8%
Not provided
Suspension reason (shown t
Suspend
Shop1 (history ↗)
Approved
83edf551b85046b8b1d57a569d0a1ebf
10%
Not provided
Suspension reason (shown t
Suspend
LibanPost eShop (history ↗)
Approved
6d0142ba41154139b68c55744136add2
5%
IBAN: LB45 0032 0000 0000 1234 5678 9012
Bank: Byblos Bank
Account name: LibanPost eShop SARL
Suspension reason (shown t
Suspend
Shops
Signed in as Store Administrator
Back to eShop

*Shops*




*Shop verification*

`/Admin/Shops` is where the marketplace's sellers are governed:

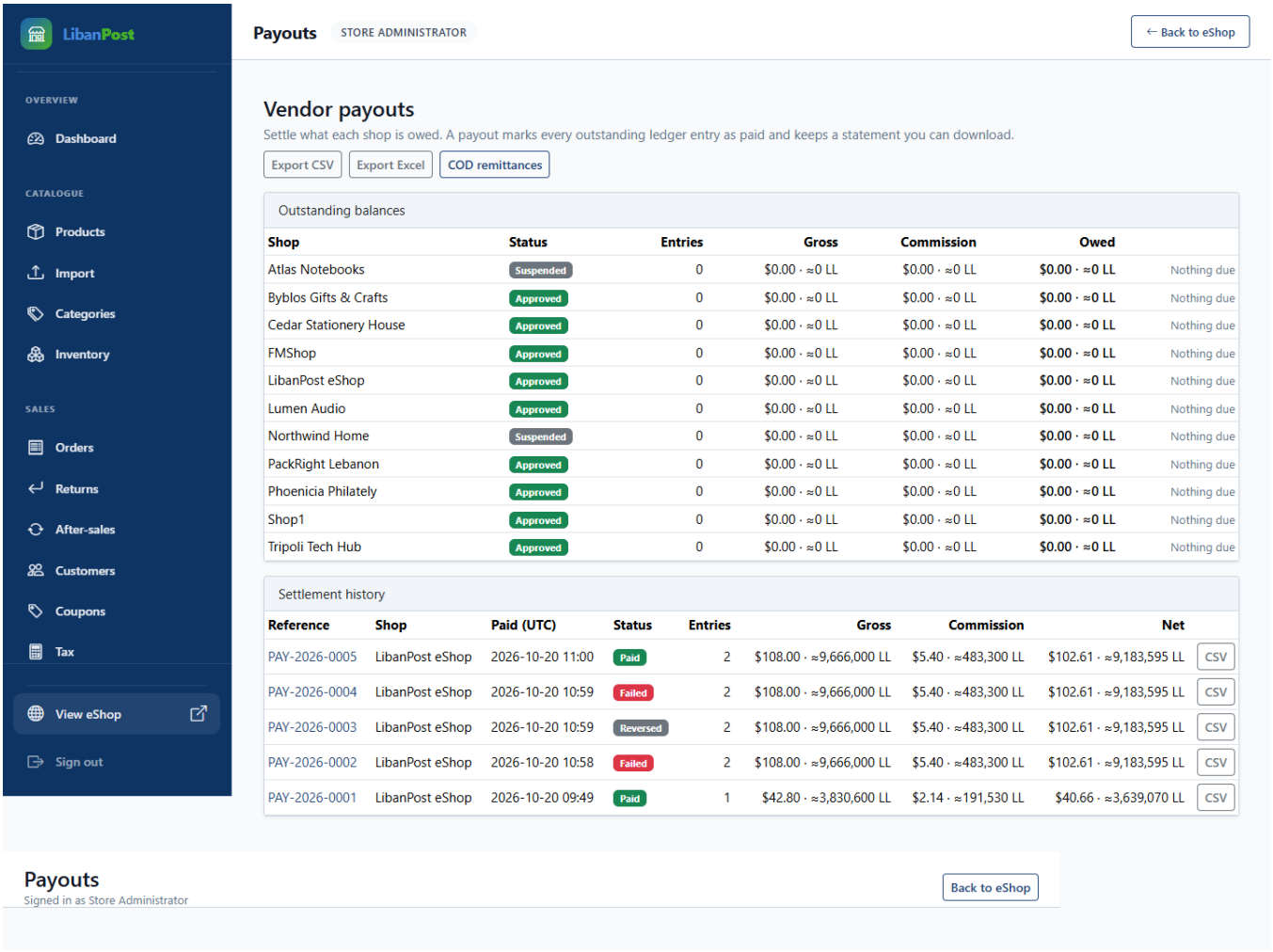
| Action | Meaning |
|---|---|
| **Approve** | The shop goes live and appears in the directory. Also where you set its **commission rate**. |
| **Reject** | Declines a pending application, with a reason the vendor sees. |
| **Suspend** | Takes a live shop off the storefront, keeping its data and workspace, with a reason. |
| **Reinstate** | Puts a suspended shop back. |

Commission is entered as a percentage in the form (`10` = 10%); the store divides by 100 to store the fraction. Commission is withheld on **goods only**, never on shipping.

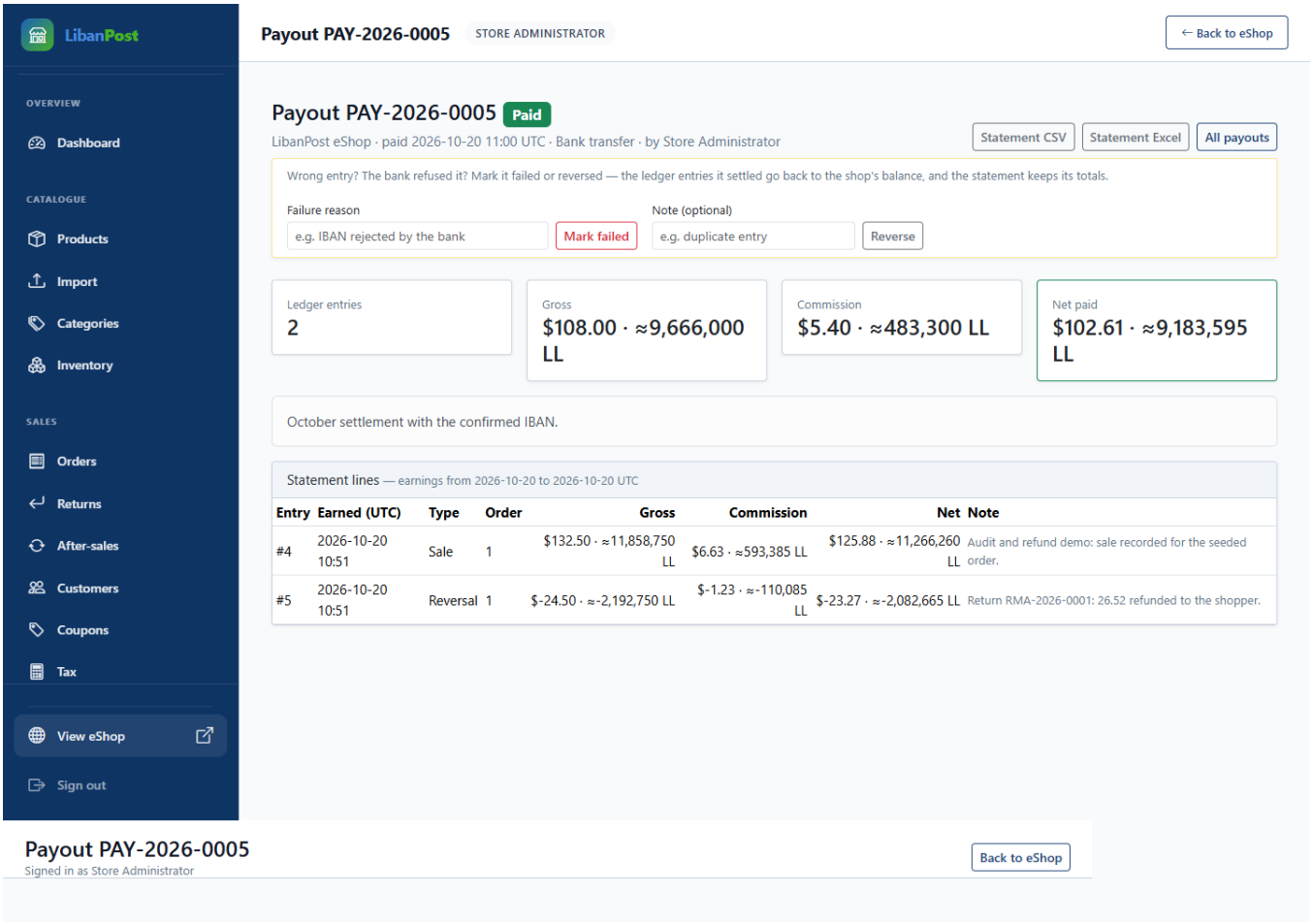
Every action on this page — approve, reject, suspend, reinstate — both **emails the shop's owner** and **writes an audit entry**, so the storefront directory's changes always have a history you can read. The list also shows each shop's recorded **payout destination** (account name, bank, IBAN) at a glance; the vendor maintains it on their payouts page or at registration. The **history** link on each row opens the audit trail pre-filtered to that shop.

---

## Payouts


LibanPost
OVERVIEW
Dashboard
CATALOGUE
Products
Import
Categories
Inventory
SALES
Orders
Returns
After-sales
Customers
Coupons
Tax
View eShop
Sign out
Payouts
STORE ADMINISTRATOR
← Back to eShop
Vendor payouts
Settle what each shop is owed. A payout marks every outstanding ledger entry as paid and keeps a statement you can download.
Export CSV
Export Excel
COD remittances
Outstanding balances
Shop | Status | Entries | Gross | Commission | Owed
Atlas Notebooks | Suspended | 0 | $0.00 · ≈0 LL | $0.00 · ≈0 LL | $0.00 · ≈0 LL | Nothing due
Byblos Gifts & Crafts | Approved | 0 | $0.00 · ≈0 LL | $0.00 · ≈0 LL | $0.00 · ≈0 LL | Nothing due
Cedar Stationery House | Approved | 0 | $0.00 · ≈0 LL | $0.00 · ≈0 LL | $0.00 · ≈0 LL | Nothing due
FMShop | Approved | 0 | $0.00 · ≈0 LL | $0.00 · ≈0 LL | $0.00 · ≈0 LL | Nothing due
LibanPost eShop | Approved | 0 | $0.00 · ≈0 LL | $0.00 · ≈0 LL | $0.00 · ≈0 LL | Nothing due
Lumen Audio | Approved | 0 | $0.00 · ≈0 LL | $0.00 · ≈0 LL | $0.00 · ≈0 LL | Nothing due
Northwind Home | Suspended | 0 | $0.00 · ≈0 LL | $0.00 · ≈0 LL | $0.00 · ≈0 LL | Nothing due
PackRight Lebanon | Approved | 0 | $0.00 · ≈0 LL | $0.00 · ≈0 LL | $0.00 · ≈0 LL | Nothing due
Phoenicia Philately | Approved | 0 | $0.00 · ≈0 LL | $0.00 · ≈0 LL | $0.00 · ≈0 LL | Nothing due
Shop1 | Approved | 0 | $0.00 · ≈0 LL | $0.00 · ≈0 LL | $0.00 · ≈0 LL | Nothing due
Tripoli Tech Hub | Approved | 0 | $0.00 · ≈0 LL | $0.00 · ≈0 LL | $0.00 · ≈0 LL | Nothing due
Settlement history
Reference | Shop | Paid (UTC) | Status | Entries | Gross | Commission | Net
PAY-2026-0005 | LibanPost eShop | 2026-10-20 11:00 | Paid | 2 | $108.00 · ≈9,666,000 LL | $5.40 · ≈483,300 LL | $102.61 · ≈9,183,595 LL | CSV
PAY-2026-0004 | LibanPost eShop | 2026-10-20 10:59 | Failed | 2 | $108.00 · ≈9,666,000 LL | $5.40 · ≈483,300 LL | $102.61 · ≈9,183,595 LL | CSV
PAY-2026-0003 | LibanPost eShop | 2026-10-20 10:59 | Reversed | 2 | $108.00 · ≈9,666,000 LL | $5.40 · ≈483,300 LL | $102.61 · ≈9,183,595 LL | CSV
PAY-2026-0002 | LibanPost eShop | 2026-10-20 10:58 | Failed | 2 | $108.00 · ≈9,666,000 LL | $5.40 · ≈483,300 LL | $102.61 · ≈9,183,595 LL | CSV
PAY-2026-0001 | LibanPost eShop | 2026-10-20 09:49 | Paid | 1 | $42.80 · ≈3,830,600 LL | $2.14 · ≈191,530 LL | $40.66 · ≈3,639,070 LL | CSV
Payouts
Signed in as Store Administrator
Back to eShop

*Payouts*




*Payout detail*

`/Admin/Payouts` is where vendor money is settled.

- The list shows every shop's **outstanding balance** — gross, commission withheld, and net — and its settlement history with a **status badge** per payout.
- **Pay out** settles the whole balance in one action, producing a statement.
- A statement, and the settlement history, can be downloaded as CSV or Excel.

What a settlement *is* matters here:

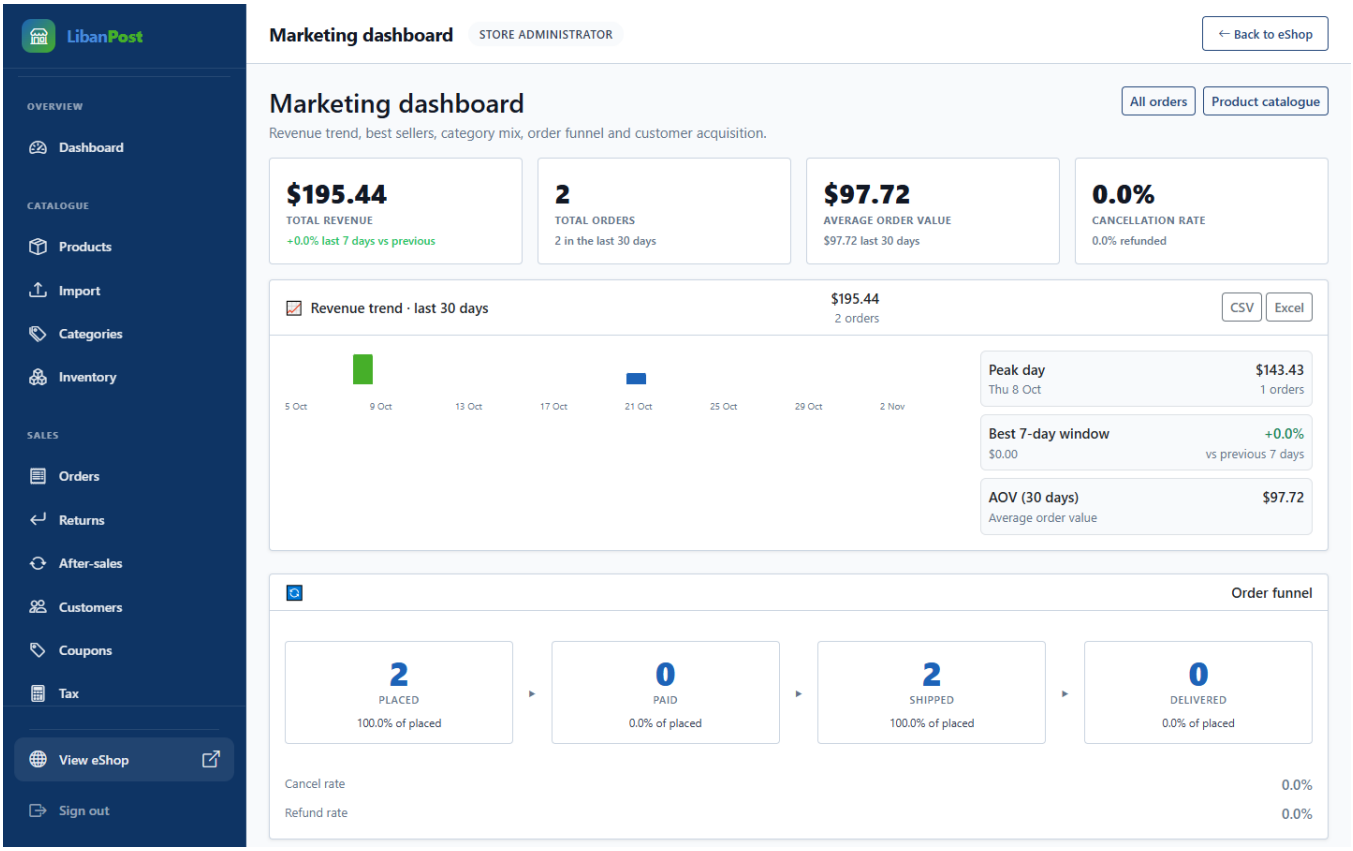
- The vendor's balance is **derived** — always the sum of their unsettled ledger entries, never a stored figure that could drift.
- A settlement **marks** those entries as settled and records a reference and period. It does not write new earnings, so a statement can always be reproduced from the ledger alone.
- **A payout is recorded, not transferred.** There is no banking integration: "Pay out" means staff moved the money by their own means and are recording it against the shop's balance.
- A balance below `Payouts:MinimumAmount` is not settled, so a shop a few cents in credit is not paid a fee to receive it.
- If a refund lands **after** a settlement, the shop's balance goes negative and no payout clears it until new sales cover it. That is the correct accounting.

**When the bank refuses a transfer** — a payout that was recorded but did not arrive is not deleted and its figures are not edited. Open it while it is still *Paid* and choose **Mark failed** with the bank's reason. The payout keeps its totals for the statement, gains a `Failed` badge with the reason, and **releases every entry it covered**

**back to the unpaid balance**, so the shop can be settled again immediately. The same release runs through **Reverse**, for a payout recorded in error. Both actions write audit entries, and the shop is emailed when a settlement is recorded against a shop with a contact email; the vendor's own payouts page shows them the badge, the reason and the handler.

---

## Marketing analytics




*Marketing analytics*

`/Admin/Marketing` answers "how are we doing?" — revenue over time, top products by revenue and by units, revenue by category, best customers, payment-method mix, coupon performance and fast-moving stock.

**Every section exports** as CSV or Excel, through the same eight section names the page presents:

`daily-revenue`, `top-products-revenue`, `top-products-units`, `revenue-by-category`, `top-customers`, `payment-methods`, `coupons`, `fast-moving`.

Two things worth knowing before you trust a figure:

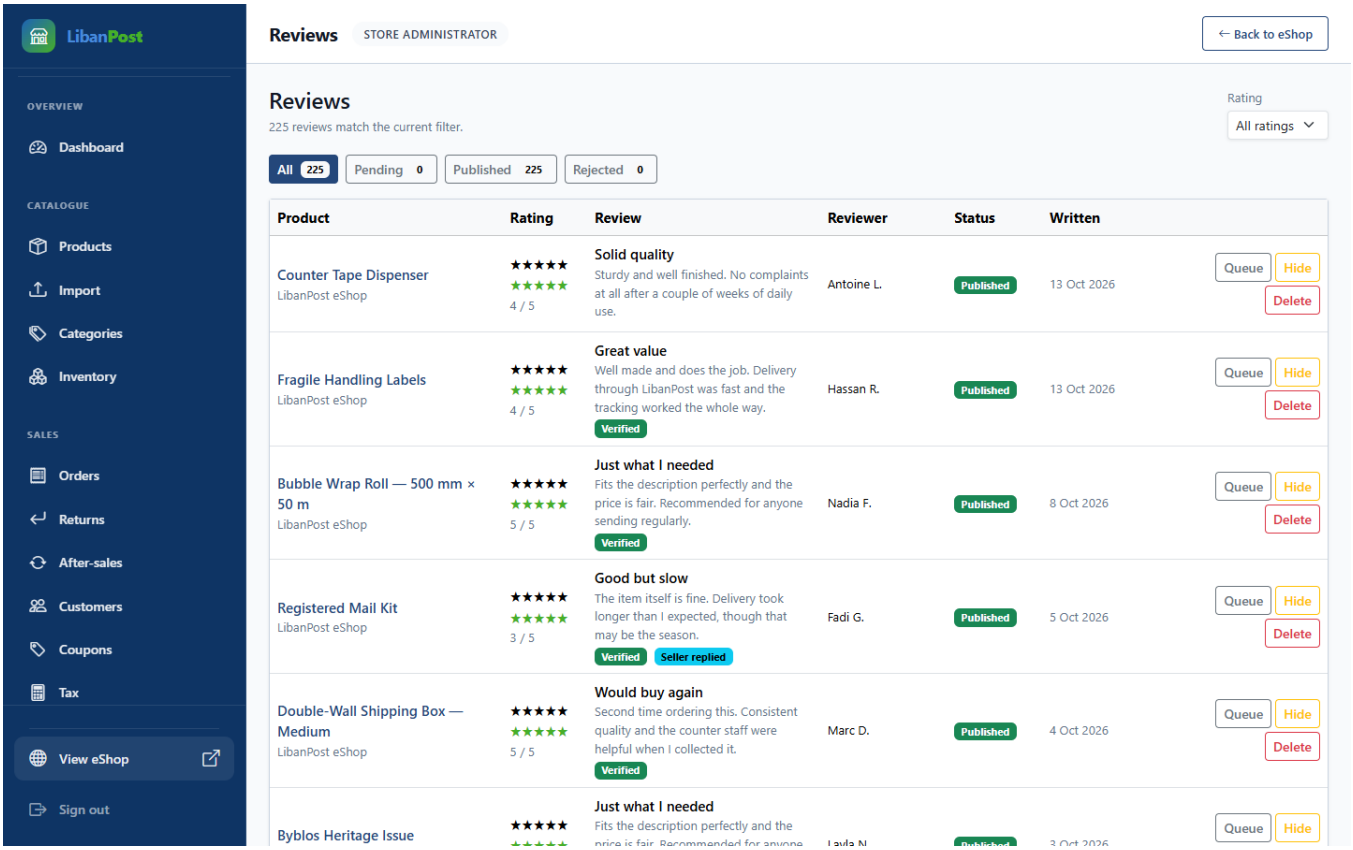
- **Only collected money counts as revenue.** A pending, cancelled or refunded order is not revenue.
- **Coupon performance reports the discount checkout actually applied**, read from the order itself,

not reconstructed from the coupon's rules.

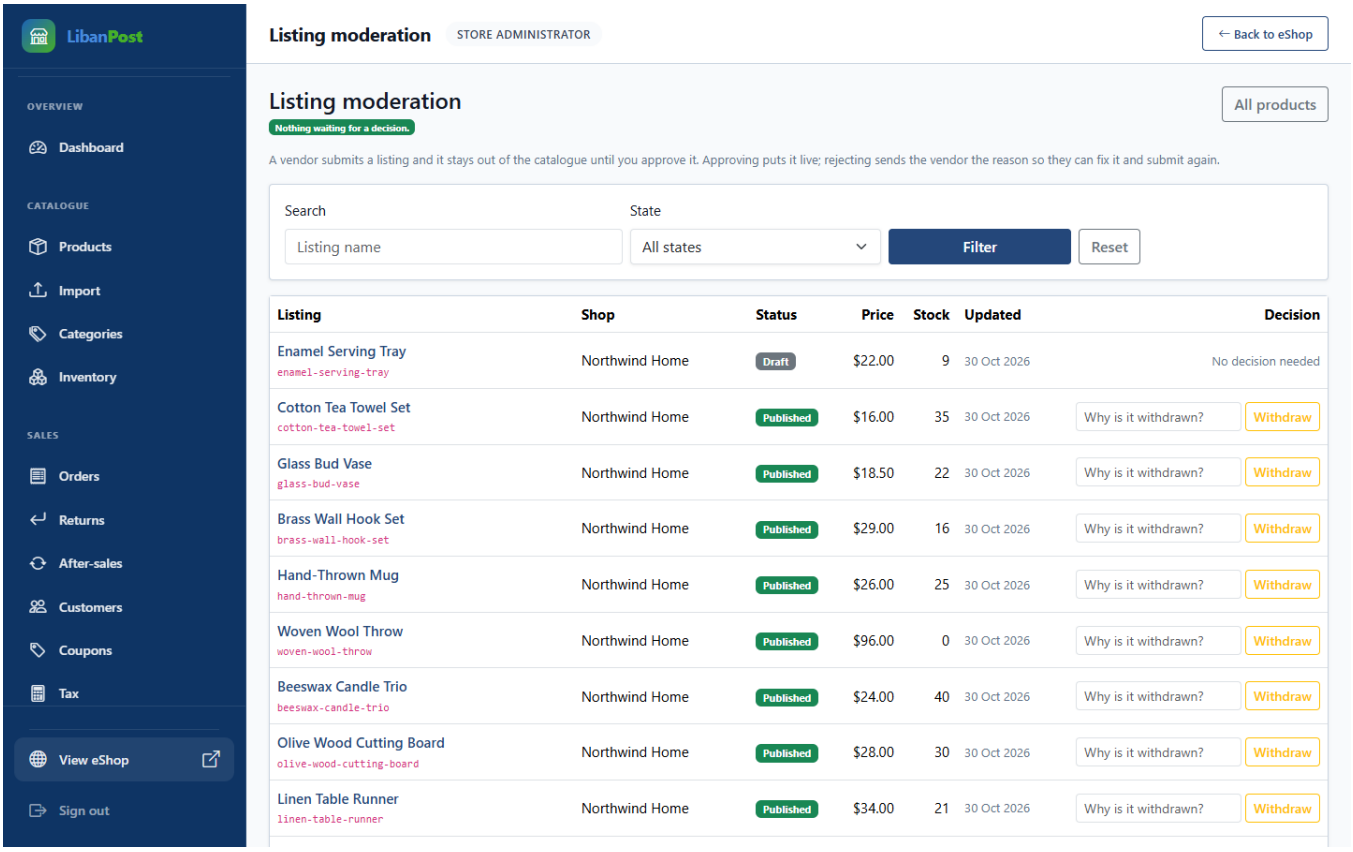
Vendors see a scoped version of the same picture on their own dashboard.

---

## Reviews moderation




*Reviews moderation*




*Moderation*

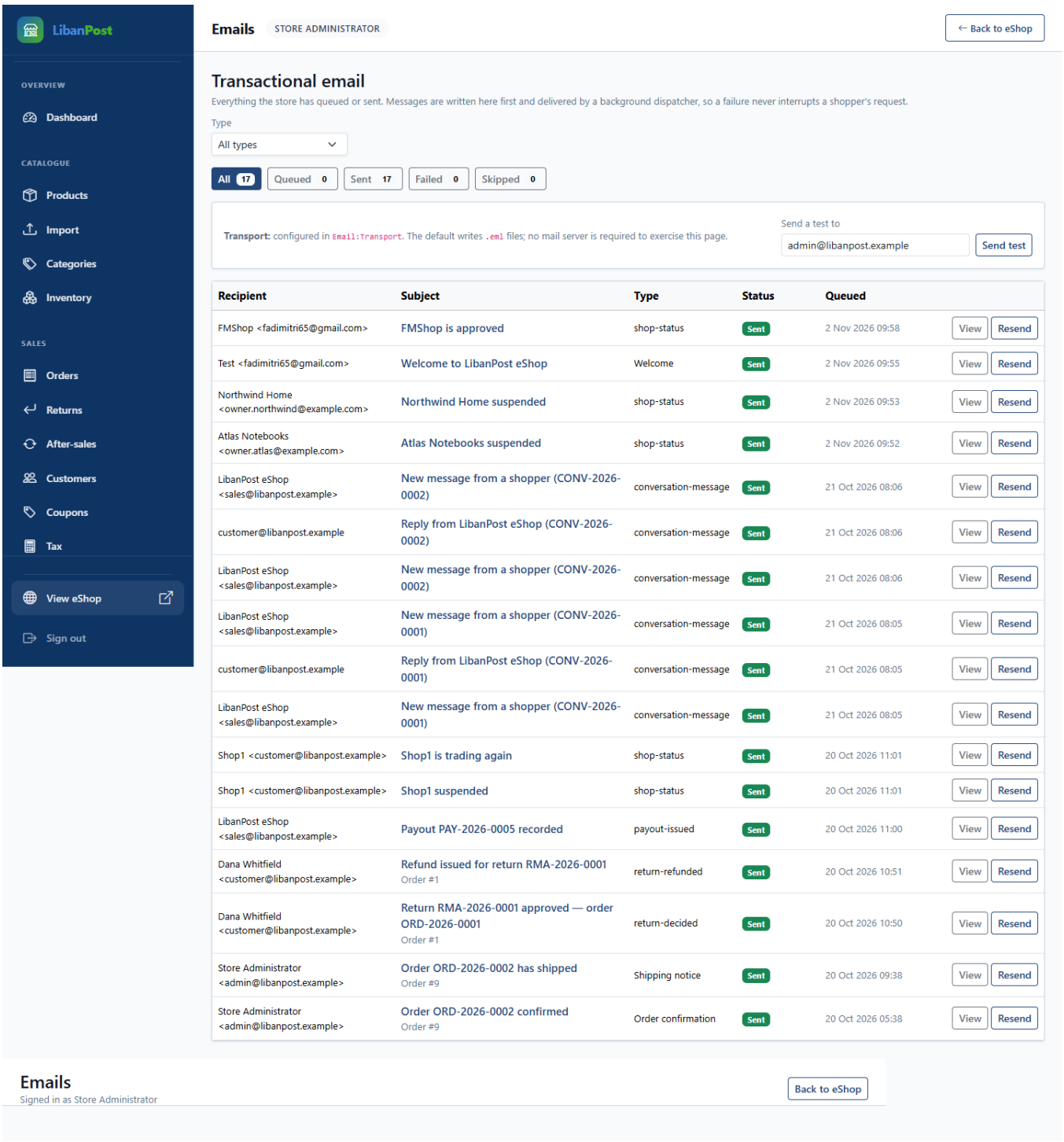
`/Admin/Reviews` is the **moderation queue**, filterable by state (pending / published / rejected) and by star rating.

Reviews **publish immediately** by default. The queue is **reactive** — it exists to hide the minority that break the rules, not to gate every review. From here you can publish, hide, return to the queue, or delete outright.

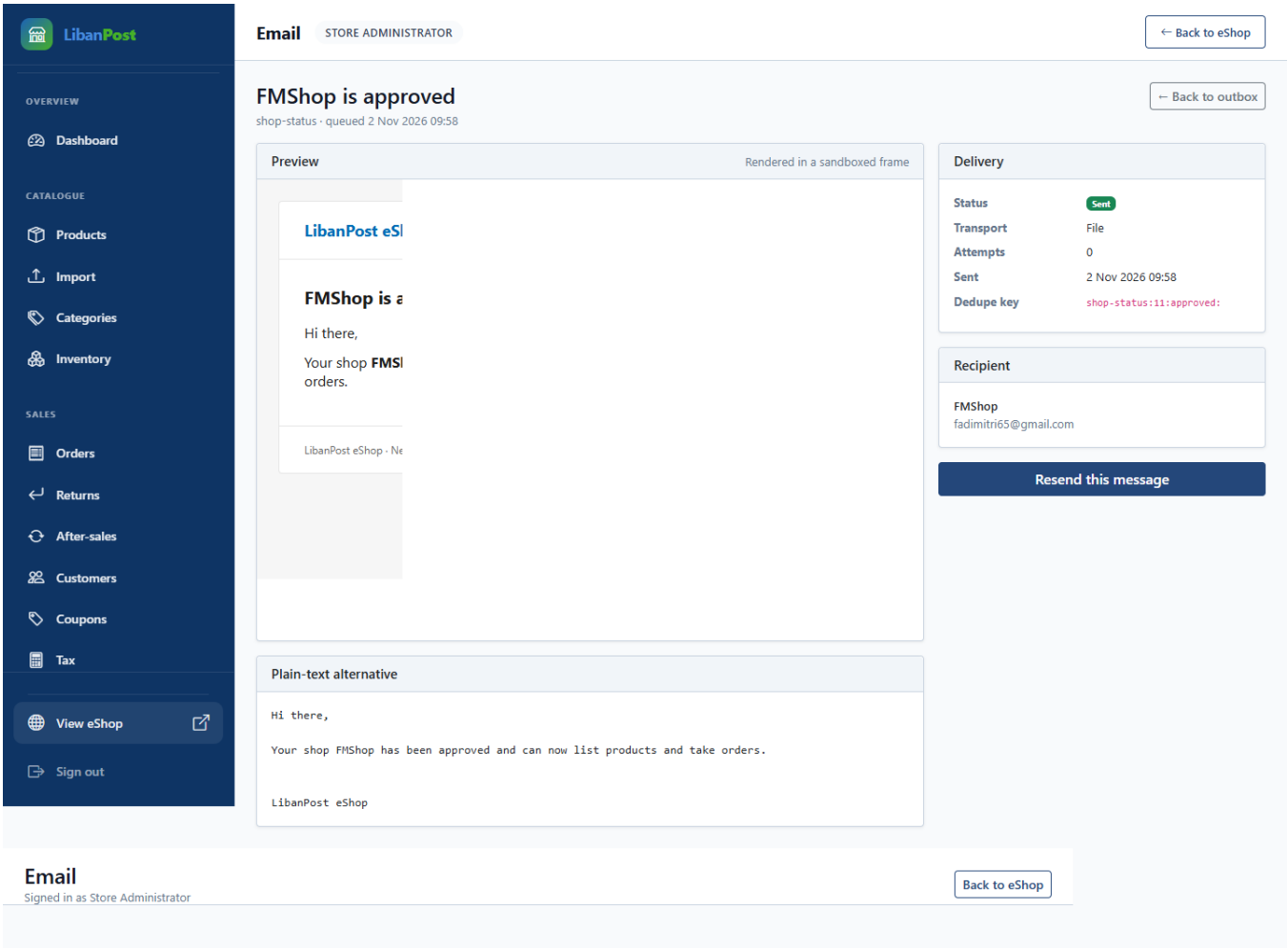
Seller replies are not moderated in the queue; a seller's reply belongs to the seller. If a reply itself breaches the rules, delete the review it sits under.

---

## Email outbox




*Email outbox*




*Email detail*

`/Admin/Emails` is the transactional outbox: every message the store **intended** to send, including the ones that failed.

- Filter by **state** (queued, sent, failed, skipped) and by **type**.
- Open a message for a **rendered preview** of what the customer receives.
- **Resend** a failed message, which returns it to the queue with a clean history.
- **Send a test message** to confirm configuration.

Design points that explain what you see:

- **Nothing sends on the request path.** A checkout queues a row and returns; a background dispatcher delivers it. A slow mail server can never slow down a checkout.
- **Messages are idempotent** by a dedupe key, so a retried request cannot send the same notification twice.
- **Failures retry with backoff**, then park as *Failed* with the full error preserved.
- With `Email:Enabled = false`, messages are recorded as **Skipped** rather than dropped, so the audit trail stays complete.
- The default transport writes real `.eml` and `.html` files instead of sending, so email works with no mail server. Nothing reaches a real inbox until an SMTP host is configured.

---

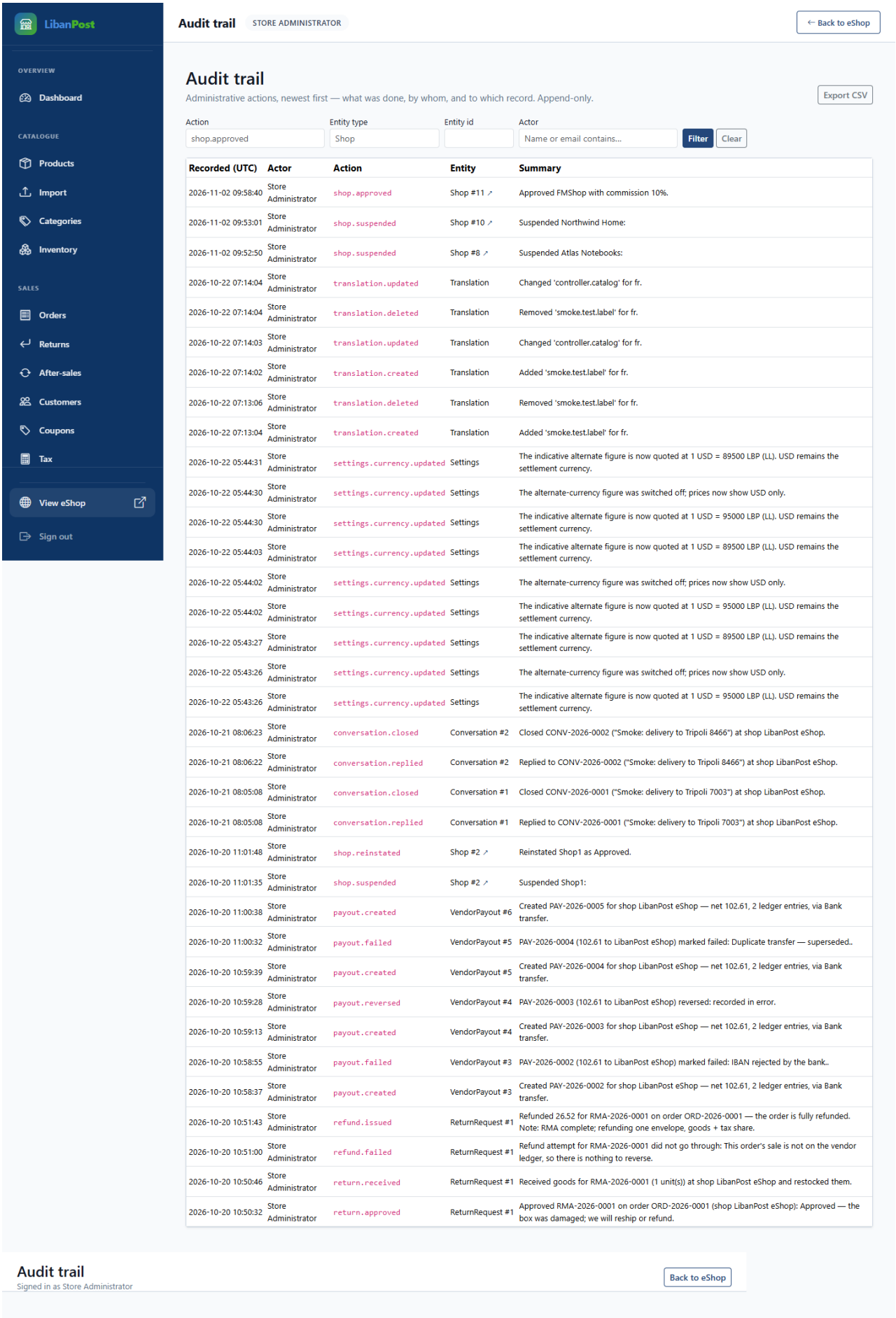
## Exports

Orders, products, customers, returns, the audit trail, payouts, tax classes and every marketing section export as **CSV** or **Excel**.

- The export runs **the same query the page does**, so the file and the screen cannot disagree about what "the current list" means — filters included.
- CSV is **UTF-8 with a byte-order mark**, so Excel reads Arabic and accented names correctly.
- A cell beginning `=`, `+`, `-` or `@` is **prefixed with an apostrophe**, so a customer whose name starts with `=` cannot smuggle a formula into your spreadsheet.
- The Excel output is a real workbook: numbers Excel can sum, real dates, a bold frozen header row and an autofilter.
- Exports are **capped** (orders 200, products 5,000) to match the page and bound memory.

---

## Audit trail


LibanPost
Audit trail
STORE ADMINISTRATOR
← Back to eShop
OVERVIEW
Dashboard
CATALOGUE
Products
Import
Categories
Inventory
SALES
Orders
Returns
After-sales
Customers
Coupons
Tax
View eShop
Sign out
Audit trail
Administrative actions, newest first — what was done, by whom, and to which record. Append-only.
Export CSV
Action
shop.approved
Entity type
Shop
Entity id
Actor
Name or email contains…
Filter
Clear
Recorded (UTC) Actor Action Entity Summary
2026-11-02 09:58:40 Store Administrator shop.approved Shop #11 ↗ Approved FMShop with commission 10%.
2026-11-02 09:53:01 Store Administrator shop.suspended Shop #10 ↗ Suspended Northwind Home:
2026-11-02 09:52:50 Store Administrator shop.suspended Shop #8 ↗ Suspended Atlas Notebooks:
2026-10-22 07:14:04 Store Administrator translation.updated Translation Changed 'controller.catalog' for fr.
2026-10-22 07:14:04 Store Administrator translation.deleted Translation Removed 'smoke.test.label' for fr.
2026-10-22 07:14:03 Store Administrator translation.updated Translation Changed 'controller.catalog' for fr.
2026-10-22 07:14:02 Store Administrator translation.created Translation Added 'smoke.test.label' for fr.
2026-10-22 07:13:06 Store Administrator translation.deleted Translation Removed 'smoke.test.label' for fr.
2026-10-22 07:13:04 Store Administrator translation.created Translation Added 'smoke.test.label' for fr.
2026-10-22 05:44:31 Store Administrator settings.currency.updated Settings The indicative alternate figure is now quoted at 1 USD = 89500 LBP (LL). USD remains the settlement currency.
2026-10-22 05:44:30 Store Administrator settings.currency.updated Settings The alternate-currency figure was switched off; prices now show USD only.
2026-10-22 05:44:30 Store Administrator settings.currency.updated Settings The indicative alternate figure is now quoted at 1 USD = 95000 LBP (LL). USD remains the settlement currency.
2026-10-22 05:44:03 Store Administrator settings.currency.updated Settings The indicative alternate figure is now quoted at 1 USD = 89500 LBP (LL). USD remains the settlement currency.
2026-10-22 05:44:02 Store Administrator settings.currency.updated Settings The alternate-currency figure was switched off; prices now show USD only.
2026-10-22 05:44:02 Store Administrator settings.currency.updated Settings The indicative alternate figure is now quoted at 1 USD = 95000 LBP (LL). USD remains the settlement currency.
2026-10-22 05:43:27 Store Administrator settings.currency.updated Settings The indicative alternate figure is now quoted at 1 USD = 89500 LBP (LL). USD remains the settlement currency.
2026-10-22 05:43:26 Store Administrator settings.currency.updated Settings The alternate-currency figure was switched off; prices now show USD only.
2026-10-22 05:43:26 Store Administrator settings.currency.updated Settings The indicative alternate figure is now quoted at 1 USD = 95000 LBP (LL). USD remains the settlement currency.
2026-10-21 08:06:23 Store Administrator conversation.closed Conversation #2 Closed CONV-2026-0002 ("Smoke: delivery to Tripoli 8466") at shop LibanPost eShop.
2026-10-21 08:06:22 Store Administrator conversation.replied Conversation #2 Replied to CONV-2026-0002 ("Smoke: delivery to Tripoli 8466") at shop LibanPost eShop.
2026-10-21 08:05:08 Store Administrator conversation.closed Conversation #1 Closed CONV-2026-0001 ("Smoke: delivery to Tripoli 7003") at shop LibanPost eShop.
2026-10-21 08:05:08 Store Administrator conversation.replied Conversation #1 Replied to CONV-2026-0001 ("Smoke: delivery to Tripoli 7003") at shop LibanPost eShop.
2026-10-20 11:01:48 Store Administrator shop.reinstated Shop #2 ↗ Reinstated Shop1 as Approved.
2026-10-20 11:01:35 Store Administrator shop.suspended Shop #2 ↗ Suspended Shop1:
2026-10-20 11:00:38 Store Administrator payout.created VendorPayout #6 Created PAY-2026-0005 for shop LibanPost eShop — net 102.61, 2 ledger entries, via Bank transfer.
2026-10-20 11:00:32 Store Administrator payout.failed VendorPayout #5 PAY-2026-0004 (102.61 to LibanPost eShop) marked failed: Duplicate transfer — superseded..
2026-10-20 10:59:39 Store Administrator payout.created VendorPayout #5 Created PAY-2026-0004 for shop LibanPost eShop — net 102.61, 2 ledger entries, via Bank transfer.
2026-10-20 10:59:28 Store Administrator payout.reversed VendorPayout #4 PAY-2026-0003 (102.61 to LibanPost eShop) reversed: recorded in error.
2026-10-20 10:59:13 Store Administrator payout.created VendorPayout #4 Created PAY-2026-0003 for shop LibanPost eShop — net 102.61, 2 ledger entries, via Bank transfer.
2026-10-20 10:58:55 Store Administrator payout.failed VendorPayout #3 PAY-2026-0002 (102.61 to LibanPost eShop) marked failed: IBAN rejected by the bank..
2026-10-20 10:58:37 Store Administrator payout.created VendorPayout #3 Created PAY-2026-0002 for shop LibanPost eShop — net 102.61, 2 ledger entries, via Bank transfer.
2026-10-20 10:51:43 Store Administrator refund.issued ReturnRequest #1 Refunded 26.52 for RMA-2026-0001 on order ORD-2026-0001 — the order is fully refunded. Note: RMA complete; refunding one envelope, goods + tax share.
2026-10-20 10:51:00 Store Administrator refund.failed ReturnRequest #1 Refund attempt for RMA-2026-0001 did not go through: This order's sale is not on the vendor ledger, so there is nothing to reverse.
2026-10-20 10:50:46 Store Administrator return.received ReturnRequest #1 Received goods for RMA-2026-0001 (1 unit(s)) at shop LibanPost eShop and restocked them.
2026-10-20 10:50:32 Store Administrator return.approved ReturnRequest #1 Approved RMA-2026-0001 on order ORD-2026-0001 (shop LibanPost eShop): Approved — the box was damaged; we will reship or refund.
Audit trail
Signed in as Store Administrator
Back to eShop

*Audit trail*

`/Admin/Audit` is the store's administrative memory: who did what, to which record, and when.

- **What is recorded** — every shop decision (approved, rejected, suspended, reinstated), every order

status change staff make, every return decision, received-goods entry, refund issued **and every refund attempt that failed**, and every payout created, marked failed or reversed. Failed attempts are kept — a trail that hides the failures cannot explain the retries.

- **What a row says** — timestamp, actor (the signed-in name as shown in the admin area), a dotted

action (`shop.approved`, `refund.failed`, `payout.reversed`, …), the subject as *entity type* + *id*, and a one-sentence summary written by the action itself.

- **Filter** by action, entity type, entity id or actor; the suggestion lists come from the data, so

a new action is filterable without any configuration. Each shop row on the shops page links to this page pre-filtered to that shop.

- **Export** the filtered view as CSV or Excel (up to 500 rows).

Design notes: the trail is **append-only** and references are deliberately **loose** (a name string and a numeric id, not foreign keys) — deleting an account or a row never rewrites the history that mentions them. It is also written **best-effort**: a failing audit write is logged, never allowed to undo the action it was documenting.

---

## Where the store's own log lives

Everything the application logs goes to the console **and** to the `Log` table in the configured database. The console is gone when the window closes; the table is what a deployment keeps.

```
SELECT TOP 50 Id, Logged, Level, Logger, Message FROM [Log] ORDER BY Id DESC;
SELECT Id, Logged, Logger, Exception FROM [Log] WHERE Level = 'Error' ORDER BY Id DESC;
```

`Error` and `Critical` rows carry the **full exception and stack trace**. The log is designed so that **a logging call can never fail, block or throw a request** — callers enqueue, and a background writer batches the insert. Retention is off by default, so the table grows until retention is configured.

---

## Questions staff usually ask

**I changed a product's price — why do old orders still show the old price?** Because they should. Order lines snapshot the product name, variant, SKU and unit price at the moment of sale, so editing a product cannot rewrite history.

**Why can't I delete this tax class?** Products still reference it. Deleting would silently move them onto the store rate. The page deactivated it instead; reassign the products, then delete.

**Why is a coupon still listed after it was used up?** A redeemed coupon is deactivated rather than deleted, so order history stays honest. It will not apply.

**Why did the vendor's balance go negative?** A refund landed after a settlement had already paid them. New sales will cover it; no payout will clear it until they do.

**The bank refused a payout — can I just delete it?** No. Mark it **failed** with the bank's reason on the payout's detail page. The covered entries return to the shop's balance, the statement keeps its figures, and every step is on the audit trail.

**A shop says it refunded a customer but I see nothing.** RMA refunds are issued here, not by the shop — check `/Admin/Returns` for the request's state. If it is still *received*, the refund has not been issued yet; the audit trail shows who did what so far.

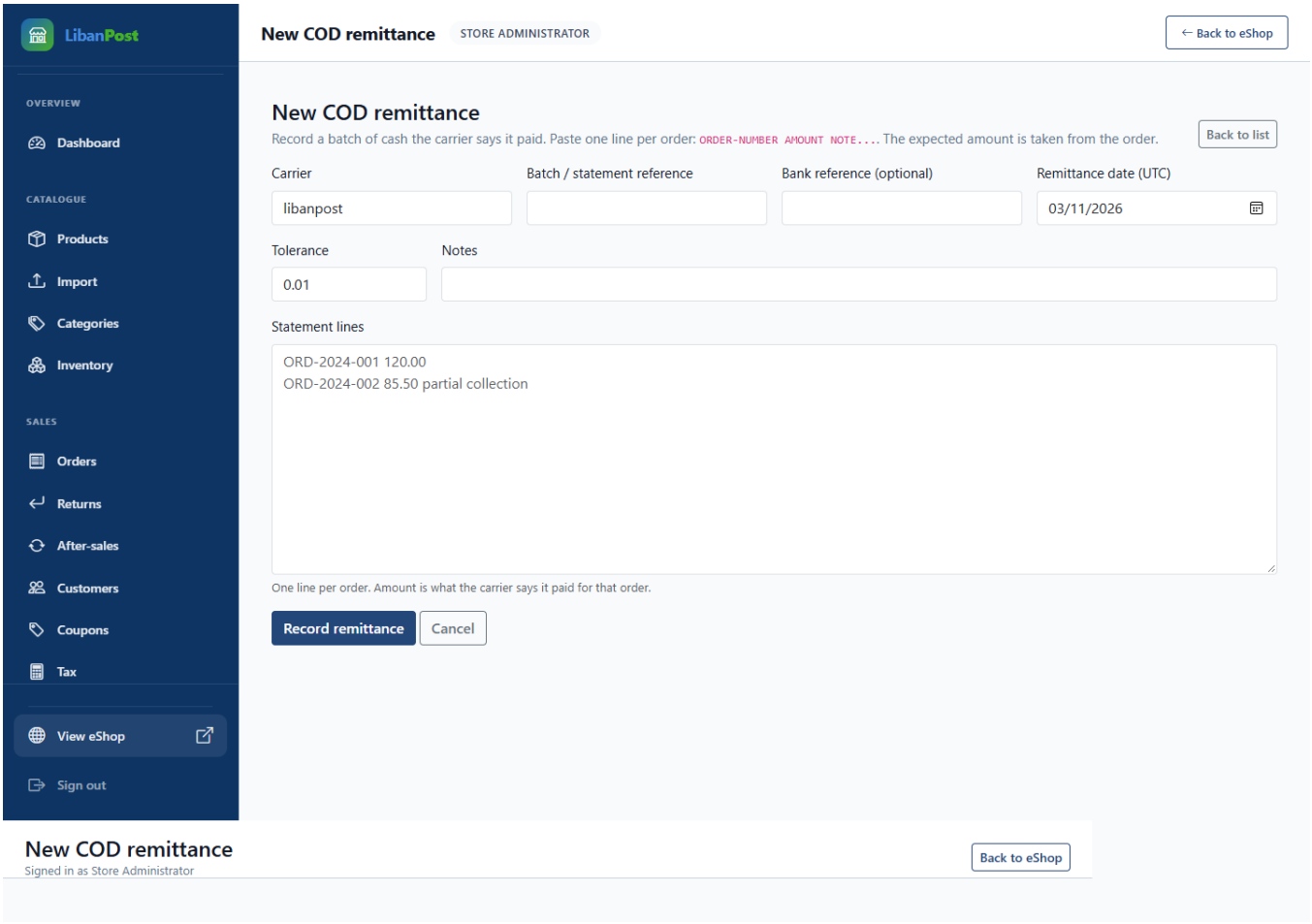
**Why does the marketing revenue differ from the order list total?** Marketing counts only **collected** money — pending, cancelled and refunded orders are excluded.

**Do coupon discounts show the right figure?** Yes. Coupon performance reports the discount the checkout actually applied, read from the order, not recalculated from the coupon's rules.

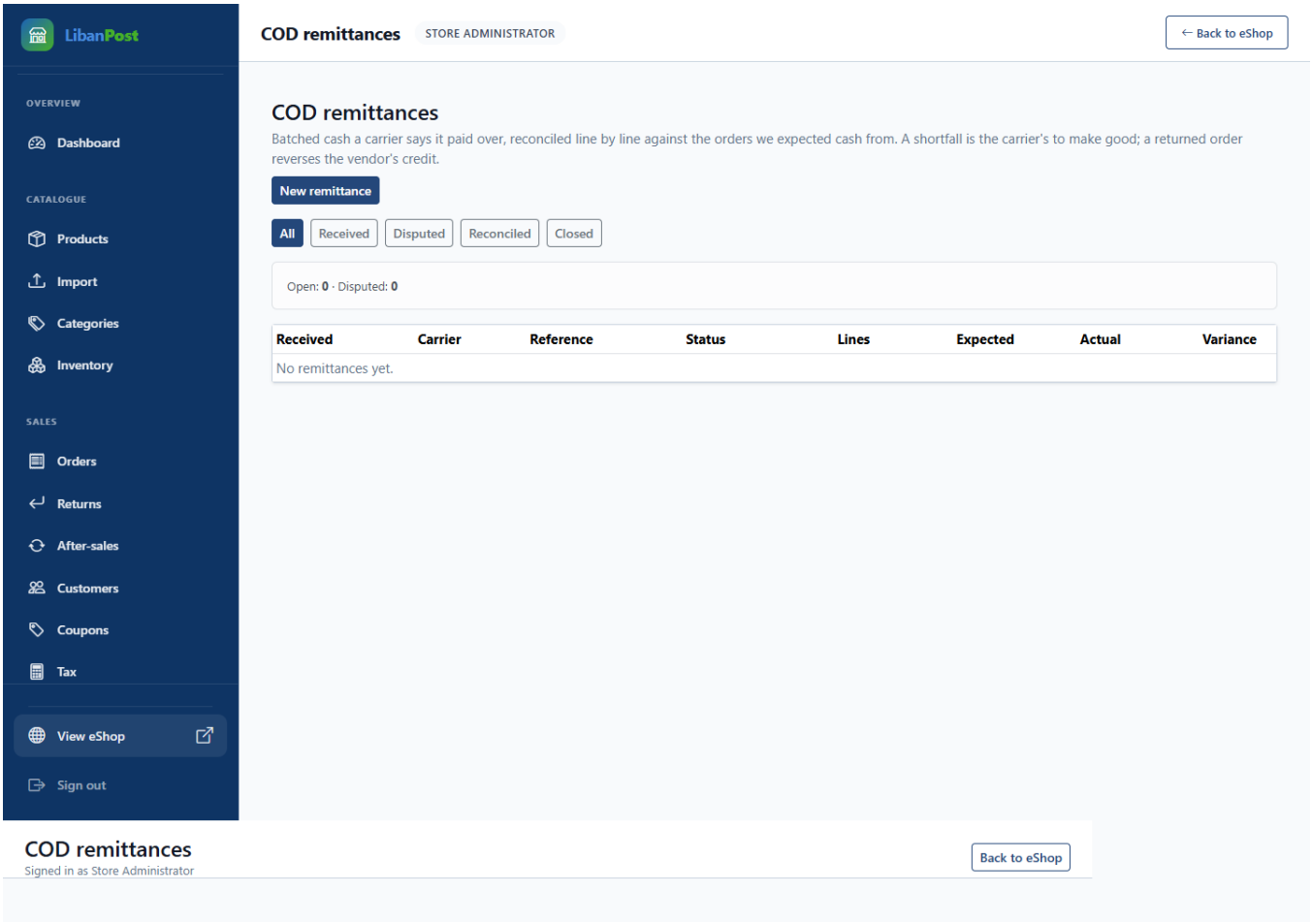
**A bulk import failed — did it write anything?** No. An import is all-or-nothing: every row is validated first, and nothing is written if any row is rejected. The errors are listed by line.

**Why is a customer's order missing from their history?** Check whether the order was placed as a **guest** with an email that is not tied to an account. Guest orders are tracked by order number and email, not by account.

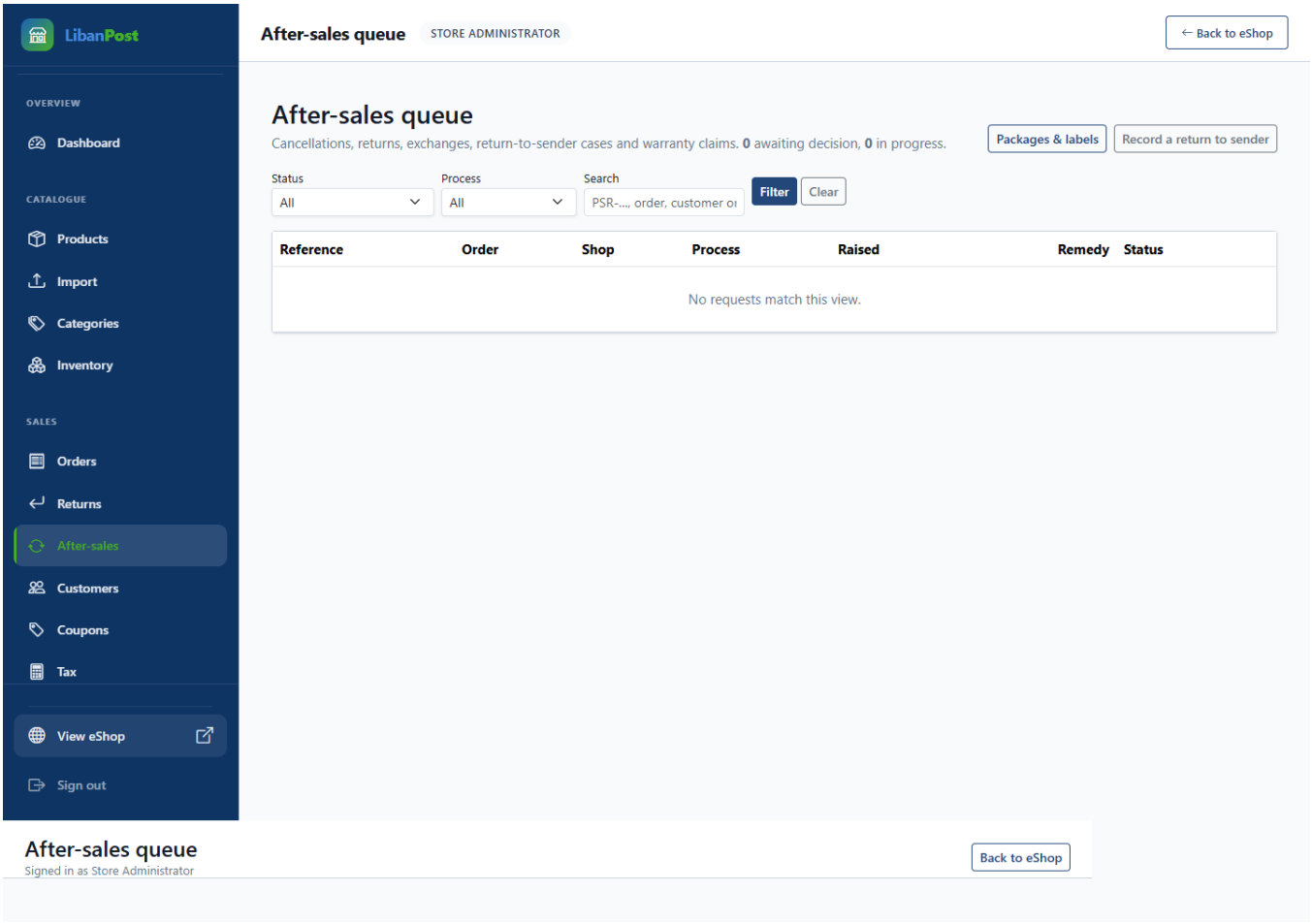
## Additional screens



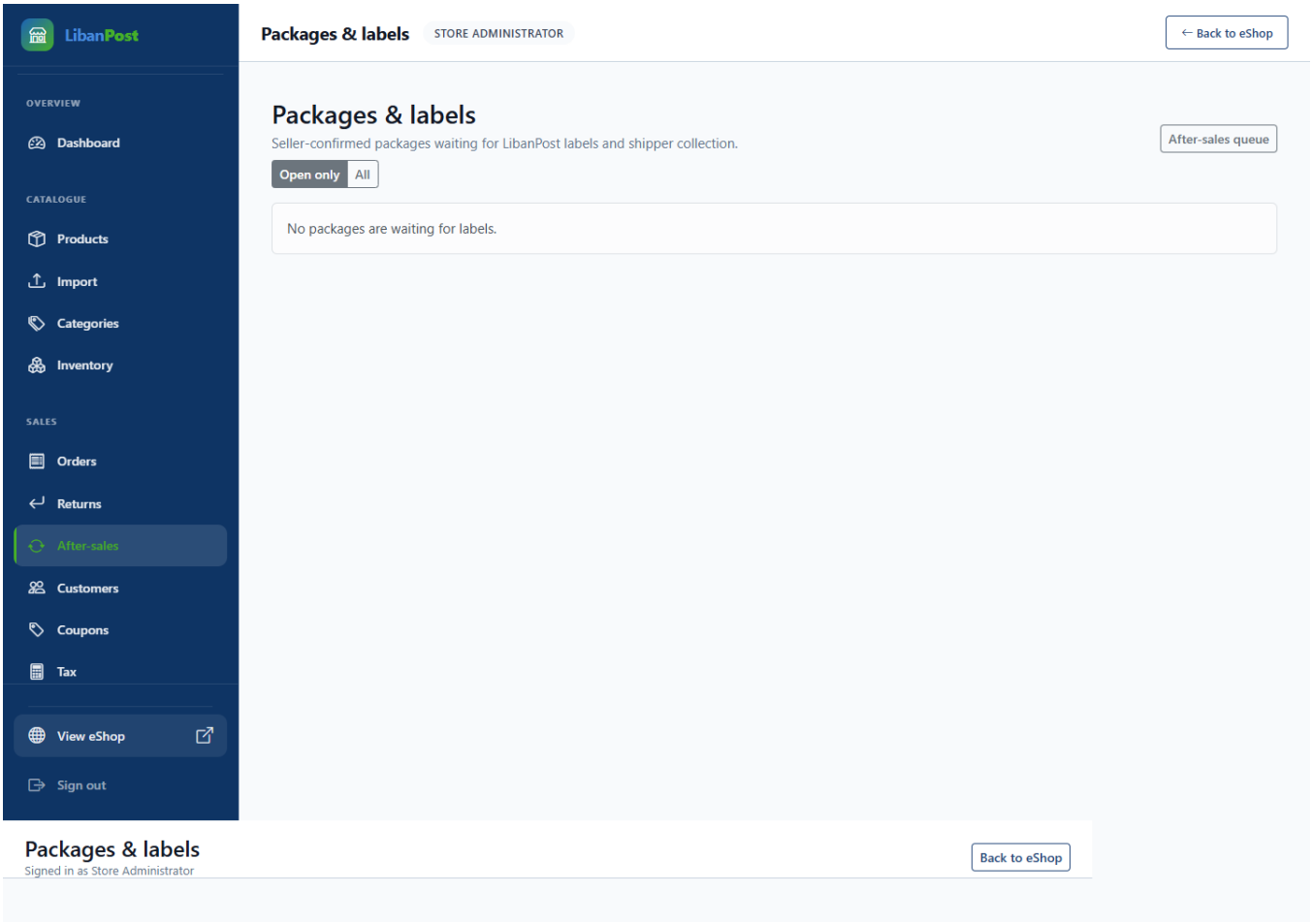

*Admin Cod Create*



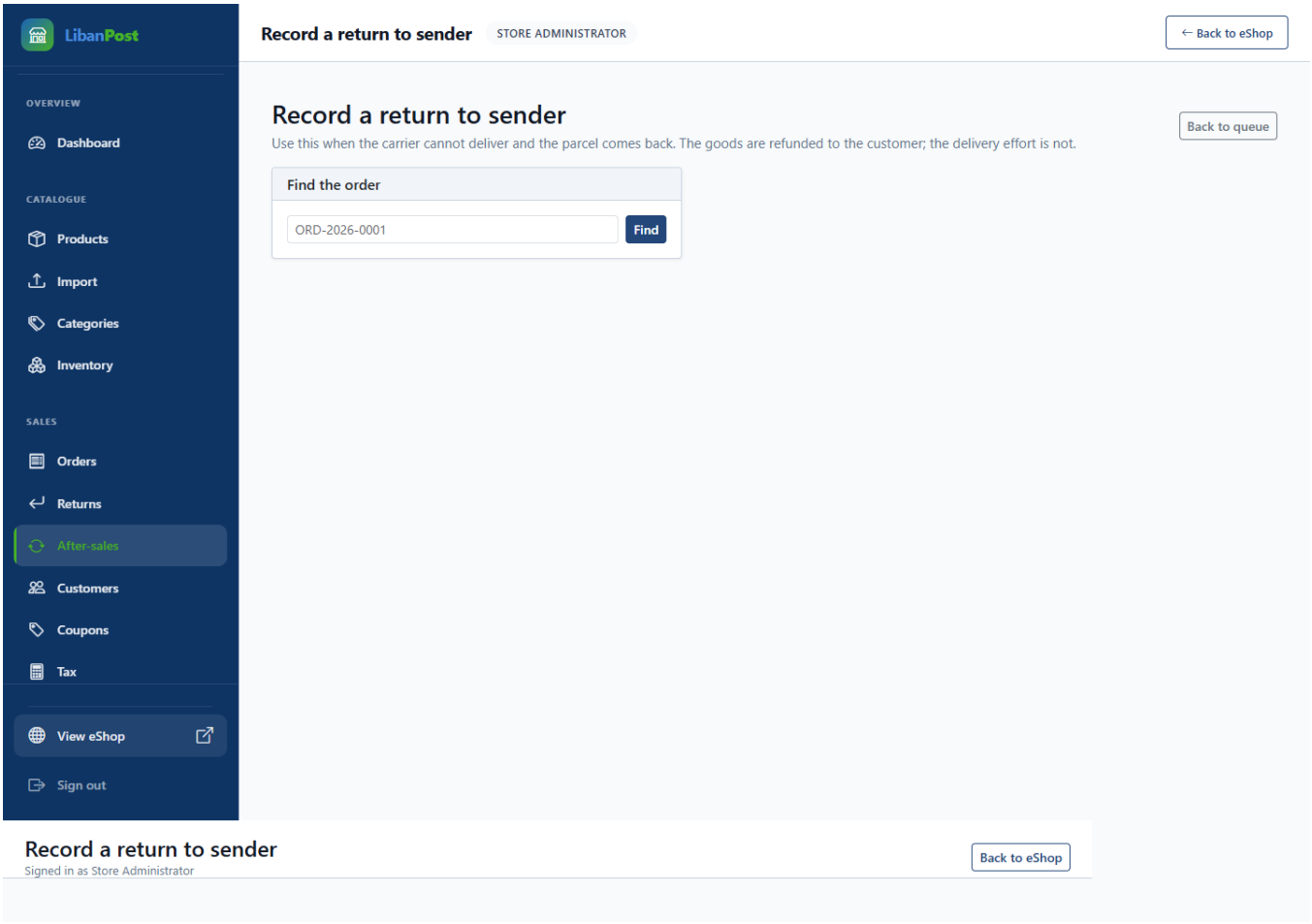

*Admin Cod Remittances*



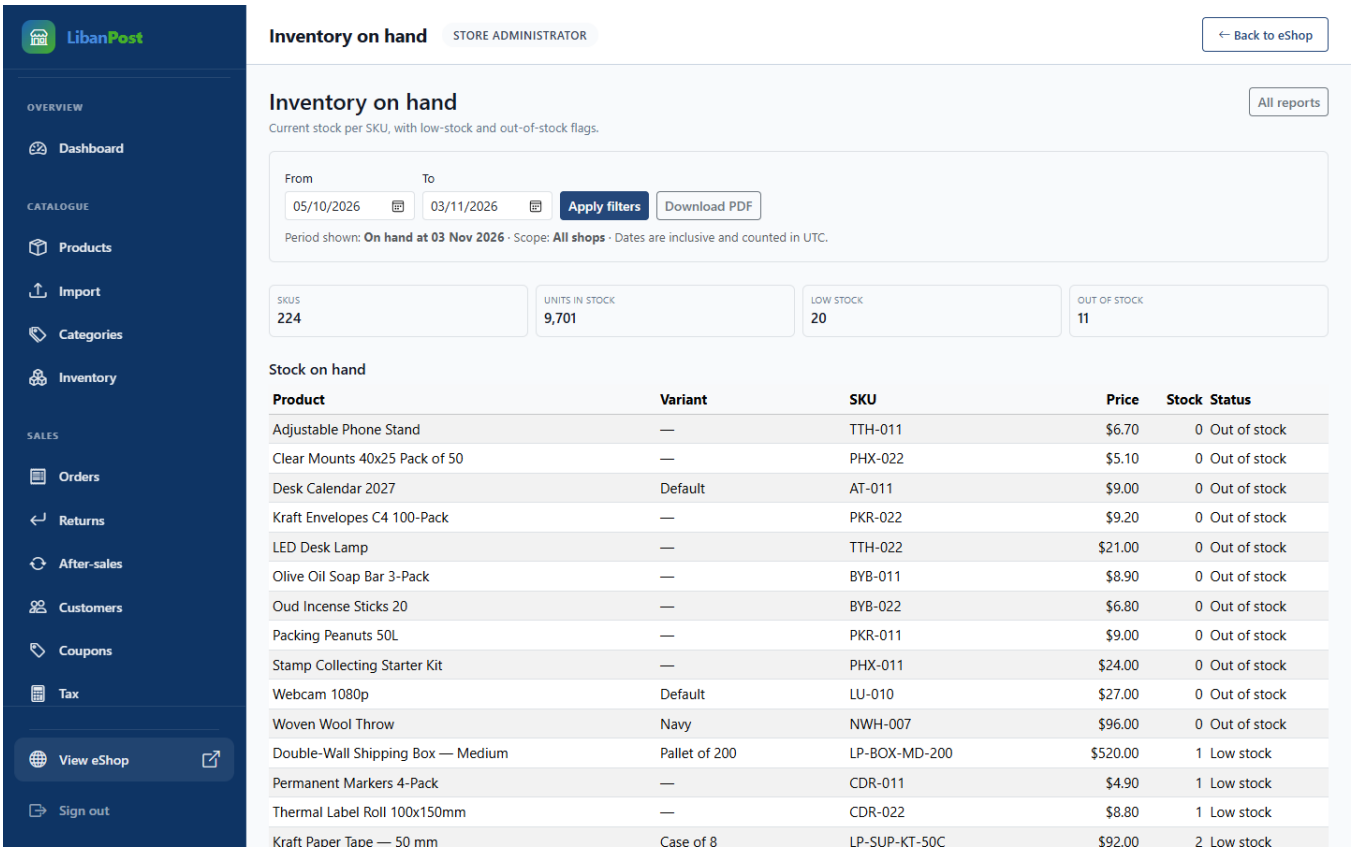

*Admin Postsale*



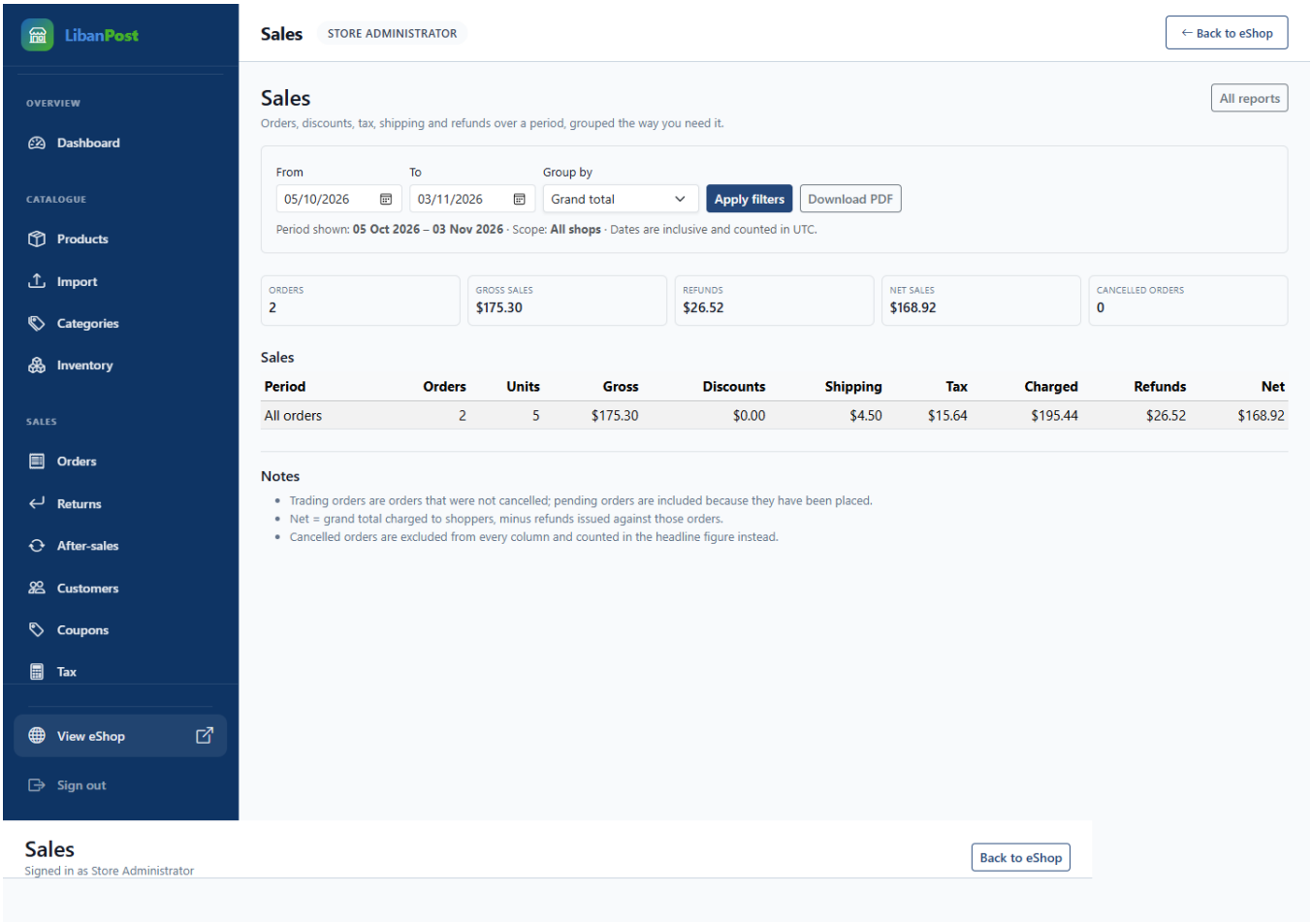

*Admin Postsale Fulfilments*



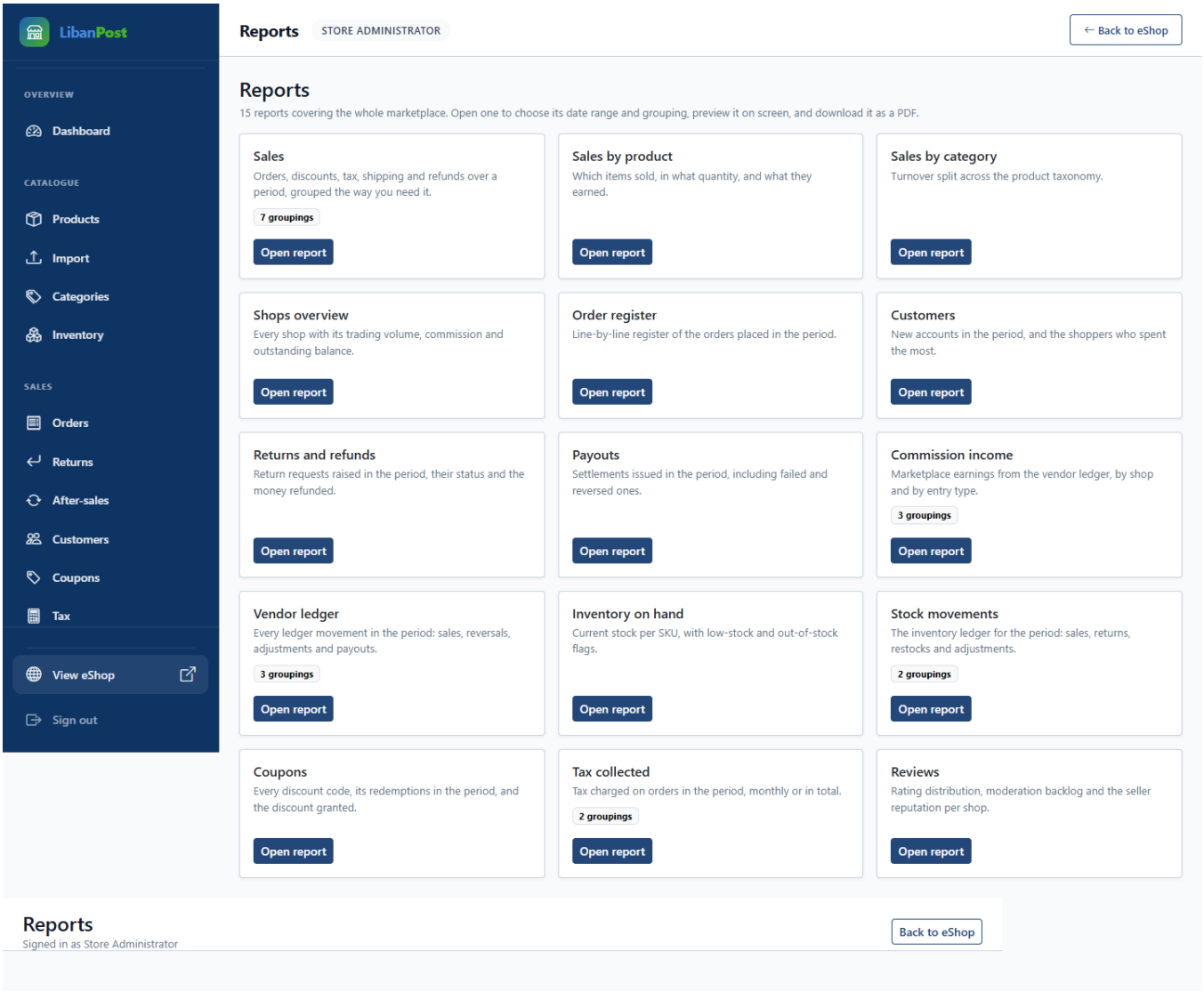

*Admin Postsale Rts*



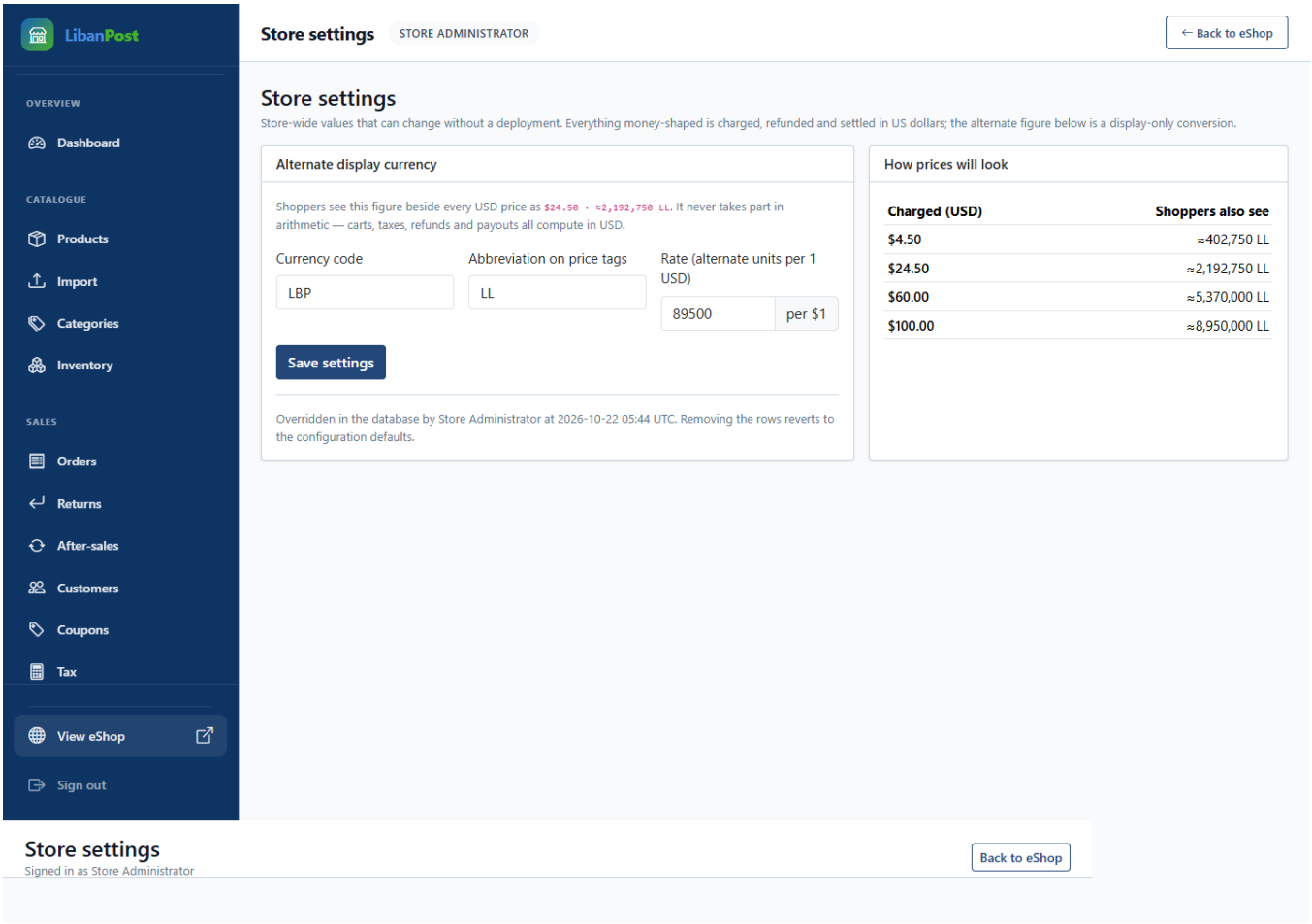

*Admin Report Inventory*



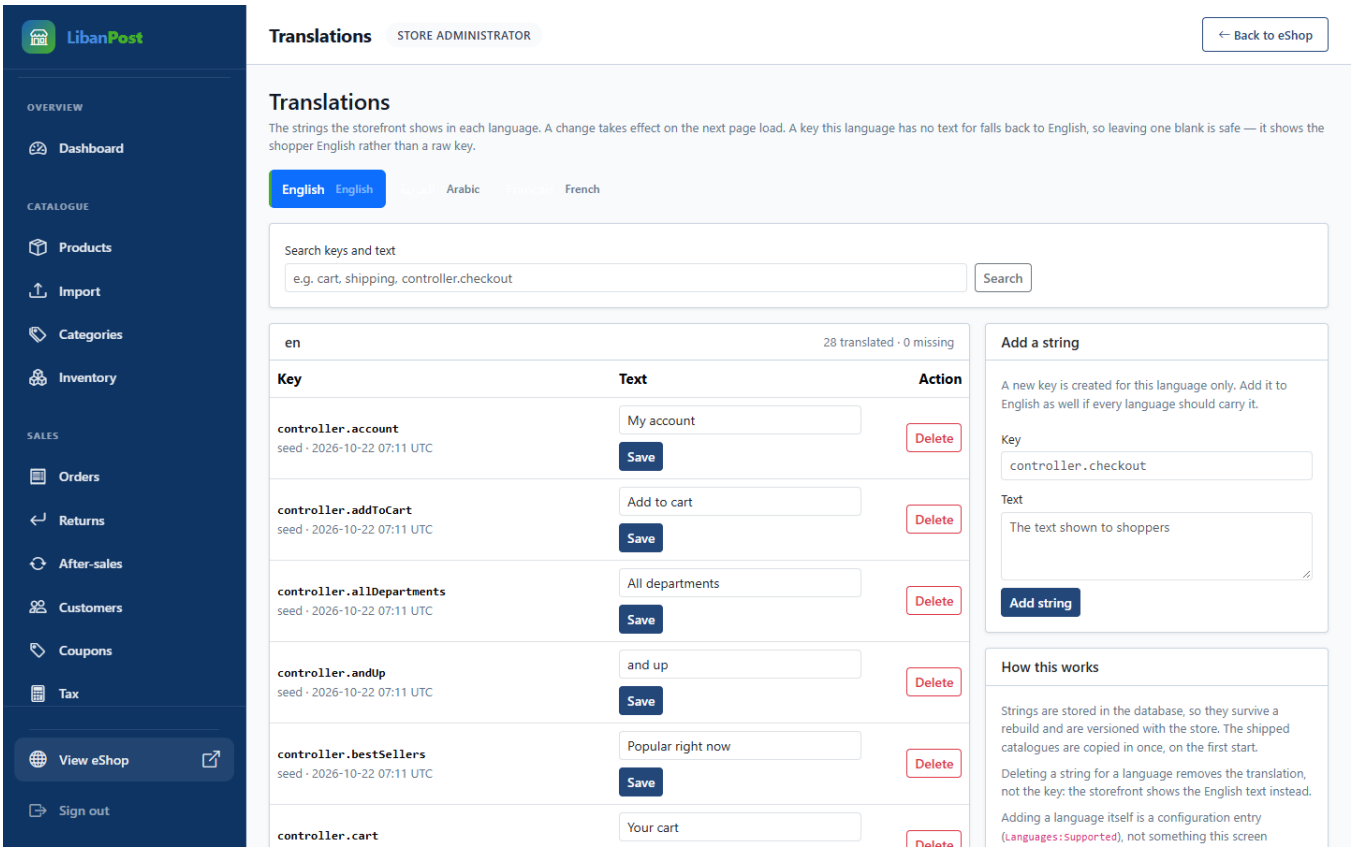

*Admin Report Sales*




*Admin Reports*




*Admin Settings*




*Admin Translations*

# Admin form and field reference

## 5. Admin area (`/Admin`)

All admin pages share `_AdminLayout` (sidebar). Antiforgery is global.

### Dashboard — `/Admin/Dashboard` (`Dashboard/Index`)

- **Model:** `DashboardViewModel`. No forms. Links to other admin pages.

### Products list — `/Admin/Products` (`Products/Index`)

- **Model:** `AdminProductListViewModel`
- **Forms:** four.

1. **Filter** (`GET Products/Index`) — `q` (search), `categoryId` (select), `state` (select).
2. **Toggle published** (`Products/TogglePublished`, POST) — `id` (hidden).
3. **Restore** (`Products/Restore`, POST) — `id` (hidden).
4. **Delete** (`Products/Delete`, POST) — `id` (hidden), confirm dialog.

### Product edit — `/Admin/Products/Edit` (and Create) (`Products/Edit` / `Save`)

- **Model:** `ProductEditViewModel`
- **Form:** one `POST` form (`Products/Save`), `enctype="multipart/form-data"`.

| Field | Type | Purpose |
|---|---|---|
| `Id` | hidden | 0 for new. |
| `Name` | input | Product name. |
| `Slug` | input | URL slug (auto-derived if blank). |
| `Brand` | input | Brand. |
| `ShortDescription` | textarea | Short description. |
| `Description` | textarea | Full description. |
| `CategoryId` | select | Category. |
| `IsPublished` | checkbox | Published. |
| `IsFeatured` | checkbox | Featured. |
| `ImageUrls` | textarea | One URL per line. |
| `ImageFiles` | file (multiple) | Uploaded images. |
| `Variants[i].Id` | hidden | Variant id (0 for new). |
| `Variants[i].Sku` | input | SKU. |
| `Variants[i].Name` | input | Variant name. |
| `Variants[i].Price` | number | Price. |
| `Variants[i].CompareAtPrice` | number | Compare-at price. |
| `Variants[i].StockQuantity` | number | Stock. |
| `Variants[i].LowStockThreshold` | number | Low-stock threshold. |
| `Variants[i].IsActive` | checkbox | Active. |
| `Variants[i].Delete` | checkbox | Mark for deletion. |

### Bulk import — `/Admin/Import` (`Import/Index`)

- **Model:** `ImportController.ImportViewModel`
- **Form:** one `POST` form, `enctype="multipart/form-data"`.

| Field | Type | Purpose |
|---|---|---|
| `shopId` | select | Target shop. |
| `file` | file | CSV file (max 5 MB). |
| `createMissing` | checkbox | Create products that do not exist yet. |

### Categories — `/Admin/Categories` (`Categories/Index`, `Save`, `Delete`)

- **Model:** `CategoryTreeViewModel` (index), `CategoryEditViewModel` (edit).
- **Forms:**

1. **Save** (`Categories/Save`, POST) — `Id` (hidden), `Name`, `Slug`, `Description`, `ParentId` (select), `DisplayOrder` (number), `IsActive` (checkbox).
2. **Delete** (`Categories/Delete`, POST) — `id` (hidden), confirm dialog.

### Inventory — `/Admin/Inventory` (`Inventory/Index`, `Adjust`, `SetStock`)

- **Model:** `InventoryViewModel`
- **Forms:** three.

1. **Filter** (`GET Inventory/Index`) — `q` (search), `filter` (select).
2. **Adjust stock** (`Inventory/Adjust`, POST) — `variantId` (hidden), `returnUrl` (hidden), `delta` (number, +/-), `reason` (select), `note` (text).
3. **Set stock** (`Inventory/SetStock`, POST) — `variantId` (hidden), `newQuantity` (number), `note` (text).

### Orders list — `/Admin/Orders` (`Orders/Index`)

- **Model:** `AdminOrderListViewModel`
- **Form:** one `GET` filter form — `q` (search), `status` (select).

### Order detail — `/Admin/Orders/Details/{id}` (`Orders/Details`)

- **Model:** `Order`
- **Form:** one `POST` form (`Orders/UpdateStatus`).

| Field | Type | Purpose |
|---|---|---|
| `id` | hidden | Order id. |
| `status` | select | New status (legal transitions only). |
| `trackingNumber` | text | Tracking number (for shipped). |

- Also shown: **Refunded so far** and **Still owed to shopper** totals under the money summary, and

the order's returns with their statuses (read-only here — the returns work at [the returns queue](#returns-queue--adminreturns-adminreturns-index)).

### Returns queue — `/Admin/Returns` (`Returns/Index`)

- **Model:** `IReadOnlyList<ReturnRequest>` (order, shop and lines included).
- **Forms:** two.

1. **Filter** (`GET Returns/Index`) — `status` (select: all plus each status), `q` (text, matches

RMA reference, shopper email or order number).

1. **Export** — GET links (`Returns/Export?format=csv|excel`), honouring the current filter.

- Shows per-status workload counters above the table; each row links to the return detail. The page

lists up to 200 rows.

### Return detail — `/Admin/Returns/Details/{id}` (`Returns/Details`)

- **Model:** `ReturnRequest` (with order, shop, lines, and the per-line product snapshots).
- **Purpose:** Act on one return. Shows the shopper's request, the shop's decision, the audit history for the return, and — depending on state — the administrator's actions: decide (approve/reject) on the shop's behalf, record the goods as received, and **Issue refund**.
- **Forms:** four (`POST`).

1. **Approve** (`Returns/Decide`) — `id` (hidden), `approve=true` (hidden), `note` (textarea, the decision note the vendor and shopper see).
1. **Reject** (`Returns/Decide`) — same shape, `approve=false`; a reason is expected.
2. **Receive** (`Returns/Receive`) — `id` (hidden); restocks the goods and opens the refund step.
3. **Refund** (`Returns/Refund`) — `id` (hidden), `note` (textarea). Issues the money: goods + proportional tax (+ shipping only for a whole-order return), capped by the ledger remainder and `GrandTotal − RefundedTotal`; refuses loudly when the sale is not on the ledger to reverse.

### Customers list — `/Admin/Customers` (`Customers/Index`)

- **Model:** `AdminCustomerListViewModel`
- **Form:** one `GET` search form — `q` (search).

### Customer detail — `/Admin/Customers/Details/{id}` (`Customers/Details`)

- **Model:** `AdminCustomerDetailViewModel`
- **Forms:** two (both `Customers/SetActive`).

1. Deactivate (confirm dialog) — `id` (hidden), `isActive` (hidden, false).
2. Activate — `id` (hidden), `isActive` (hidden, true).

### Coupons list — `/Admin/Coupons` (`Coupons/Index`)

- **Model:** `IReadOnlyList<Coupon>`
- **Form:** one `POST` per coupon (`Coupons/Delete`) — `id` (hidden), confirm dialog.

### Coupon create — `/Admin/Coupons/Create` (`Coupons/Create`)

- **Model:** `Coupon`
- **Form:** one `POST` form (`Coupons/Create`).

### Coupon edit — `/Admin/Coupons/Edit/{id}` (`Coupons/Edit`)

- **Model:** `Coupon`
- **Form:** one `POST` form (`Coupons/Edit`).

The shared partial `_Form.cshtml` (used by both create and edit) exposes:

| Field | Type | Purpose |
|---|---|---|
| `Id` | hidden | 0 for new. |
| `Code` | input | Coupon code. |
| `Type` | select | Percentage or fixed amount. |
| `Value` | number | Discount value. |
| `MinSubtotal` | number | Minimum order subtotal. |
| `MaxUses` | number | Global usage cap. |
| `MaxUsesPerUser` | number | Per-shopper cap. |
| `StartsUtc` | datetime-local | Valid from. |
| `ExpiresUtc` | datetime-local | Valid until. |

| Field | Type | Purpose |
|---|---|---|
| `Description` | input | Internal description. |
| `AppliesToAll` | checkbox | Applies to all shops. |
| `ShopId` | select | Restrict to one shop. |
| `IsActive` | checkbox | Active. |

### Tax classes — `/Admin/Tax` (`Tax/Index`, `Create`, `Deactivate`)

- **Model:** `TaxController.TaxListViewModel`
- **Forms:** two.

1. **Create** (`Tax/Create`, POST) — `name` (input), `rate` (number, 0–1), `isExempt` (checkbox).
2. **Deactivate** (`Tax/Deactivate`, POST) — `id` (hidden).

### Shops — `/Admin/Shops` (`Shops/Index`)

- **Model:** `IReadOnlyList<Shop>`
- **Forms:** four per shop row.

1. **Approve** (`Shops/Approve`, POST) — `id` (hidden), `commissionPercent` (number, e.g. 10).
2. **Reject** (`Shops/Reject`, POST) — `id` (hidden), `reason` (shown to the vendor).
3. **Suspend** (`Shops/Suspend`, POST) — `id` (hidden), `reason` (shown to the vendor).
4. **Reinstate** (`Shops/Reinstate`, POST) — `id` (hidden).

- The table also shows each shop's recorded **payout destination** (account name, bank, IBAN, or "Not provided") and a **history** link straight to `/Admin/Audit?entityType=Shop&entityId={id}`.

### Payouts list — `/Admin/Payouts` (`Payouts/Index`)

- **Model:** `AdminPayoutIndexViewModel`
- **Form:** one `POST` form (`Payouts/Create`) — `shopId` (hidden), `method` (input), `note` (input).
- The balances table shows what each shop is owed; the history table carries a **status** column (`Paid` / `Failed` / `Reversed` badges).

### Payout detail — `/Admin/Payouts/Details/{id}` (`Payouts/Details`)

- **Model:** `AdminPayoutDetailViewModel`.
- Read-only statement (reference, period, every ledger entry it settled), plus, **while the payout is still `Paid`**, two correction forms (both `POST`, `id` hidden):

1. **Mark failed** (`Payouts/MarkFailed`) — `reason` (required, ≤ 500 chars); the bank refused. Releases every covered entry back to the unpaid balance.
1. **Reverse** (`Payouts/Reverse`) — `note` (optional); recorded in error. The same release, with the row keeping its totals for the statement. A `Failed`/`Reversed` payout shows its reason, who handled it, and when — and no forms.

### Marketing — `/Admin/Marketing` (`Marketing/Index`)

- **Model:** `MarketingDashboardStats`. No forms (exports are GET links).

### Reviews moderation — `/Admin/Reviews` (`Reviews/Index`)

- **Model:** `AdminReviewListViewModel`
- **Forms:** five.

1. **Filter** (`GET Reviews/Index`) — `status` (hidden), `ratingFilter` (select).

2. Three **SetStatus** forms (`Reviews/SetStatus`, POST) — `id`, `status`, `filter`, `rating`, `page` (hidden).
3. **Delete** (`Reviews/Delete`, POST) — `id`, `filter`, `rating`, `page` (hidden), confirm dialog.

### Audit trail — `/Admin/Audit` (`Audit/Index`, `Export`)

- **Model:** `IReadOnlyList<AuditLog>` (newest first, 200 rows per page).
- **Forms:** two.

1. **Filter** (`GET Audit/Index`) — `actionName` (text with datalist suggestions derived from the data), `entityType` (datalist), `entityId` (number), `actor` (text).
1. **Export** — GET links (`Audit/Export?actionName=…&entityType=……`, up to 500 rows, `?format=csv|excel`).

- Each row: timestamp, actor, dotted action (`shop.approved`, `refund.failed`, …), entity reference and the summary sentence. Per-entity links (e.g. from the shops list) land here pre-filtered.

### Emails outbox — `/Admin/Emails` (`Emails/Index`, `Details`, `Resend`, `SendTest`)

- **Model:** `AdminEmailListViewModel` (list), `EmailMessage` (detail).
- **Forms:**

1. **Filter** (`GET Emails/Index`) — `status` (hidden), `kindFilter` (select).
2. **Send test** (`Emails/SendTest`, POST) — `status` (hidden), `kind` (hidden), `page` (hidden), `testTo` (email).
3. **Resend** (`Emails/Resend`, POST) — `id`, `status`, `kind`, `page` (hidden).

- Email detail page has a **Resend** form — `id` (hidden).

---

# Common buttons and what they do (plain language)

| Button / action | What it does |
|---|---|
| **Save / Create / Add** | Writes the form to the database. If a required field is missing, the page re-opens with a red message and nothing is saved. |
| **Cancel** | Throws away what you typed and returns to the previous page. No data is saved. |
| **Edit / pencil icon** | Opens the same record in an editable form. |
| **Delete / trash icon** | Removes the record. Where deletion would break history (orders, coupons already used, tax classes in use) the store deactivates instead, so old references keep working. |
| **Search / Filter** | Re-runs the list with the chosen filters. Filters live in the web address, so you can bookmark or share a filtered view. |
| **Export CSV / Export Excel** | Downloads the current filtered list as a file. |
| **Publish / Unpublish** | Shows or hides a product on the storefront. Drafts stay invisible. |
| **Approve / Reject** | Decides a pending shop application or return request. Reject needs a reason, which the customer sees. |
| **Suspend / Reinstate** | Takes a shop off the storefront (keeping its data) or brings it back. |
| **Pay out** | Settles a vendor's whole outstanding balance and produces a downloadable statement. |
| **Mark failed / Reverse** (payouts) | Records that a bank transfer did not go through (with the bank's reason) or was recorded in error. Releases the covered earnings back to the unpaid balance. |
| **Issue refund** | Computes and records a refund for a received return. Only store staff can do this. |
| **Record goods received** | Puts returned units back on the shelf and writes a return-stamped stock movement. |
| **Resend** (emails) | Returns a failed email to the queue with a clean history. |
| **Send test** | Queues a test message to confirm email configuration. |
| **Toggle wishlist / heart** | Adds or removes a product from your wishlist. |
| **Request a return** | Opens the return form for an order. |
| **Cancel request** | Withdraws a return you raised, before the goods are received. |